NITTE (Deemed to be University) | NMAM INSTITUTE OF TECHNOLOGY

Applied Digital Logic Design (ECE101)

Dr.Chaitra S N
Department of ISE
N.M.A.M. Institute of Technology, Nitte

Syllabus

UNIT – III

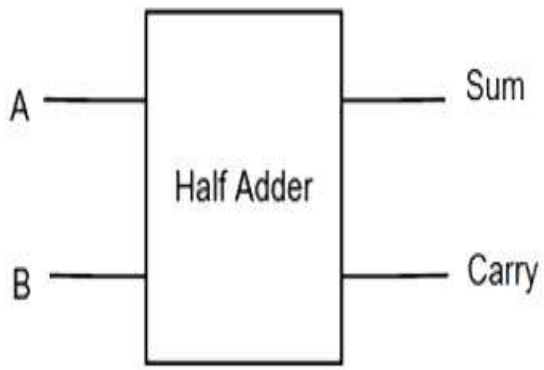
Combinational Logic Circuits: Introduction to Combinational Logic Circuits, Half/Full Adders/Subtractors, Parallel Adders/Subtractors, Binary Comparators, Decoders, Encoders,Multiplexers.

06Hrs

Combinational Logic Circuits

Half Adder

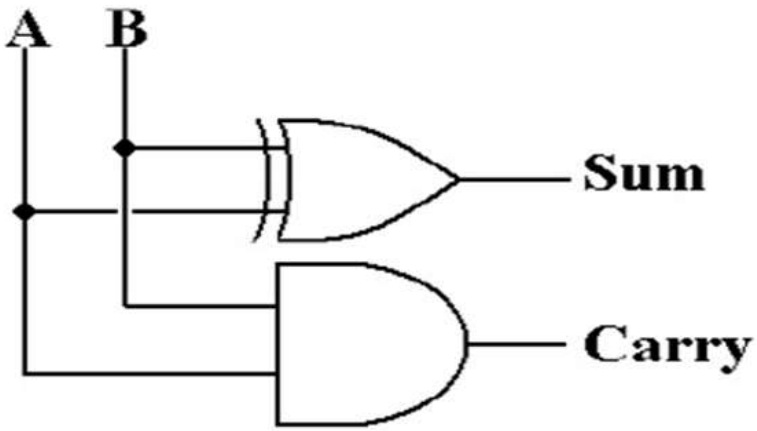
Half Adder adds to one-bit numbers and produces two outputs sum and Carry



A	B	Sum	Carry
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

Sum = A'.B + A.B' = A XOR B

Carry = A.B



Full Adder

A full adder is a combinational logic circuit that performs the addition of three single-bit numbers.

It is a digital circuit that has three inputs A, B, and C_{in} , where C_{in} is the previous carry and two output sum (S), carry (C_{out}).

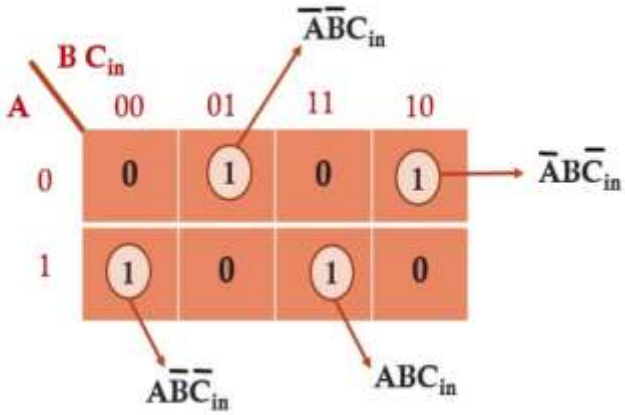
Here sum is the least significant bit (LSB) and carry is the most significant bit (MSB).

A full adder circuit is used in computer ALU (Arithmetic and Logic Unit) to perform arithmetic operations.

Inputs			Outputs	
A	B	C_{in}	S	C_{out}
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

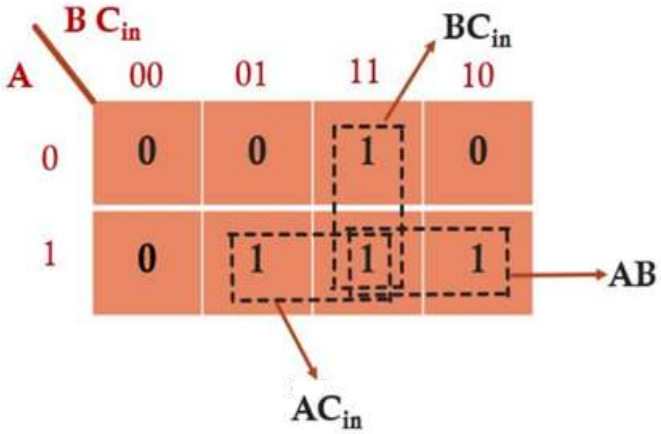
Full Adder Truth Table

K-Map for the Sum Output



k-map for Sum (S)

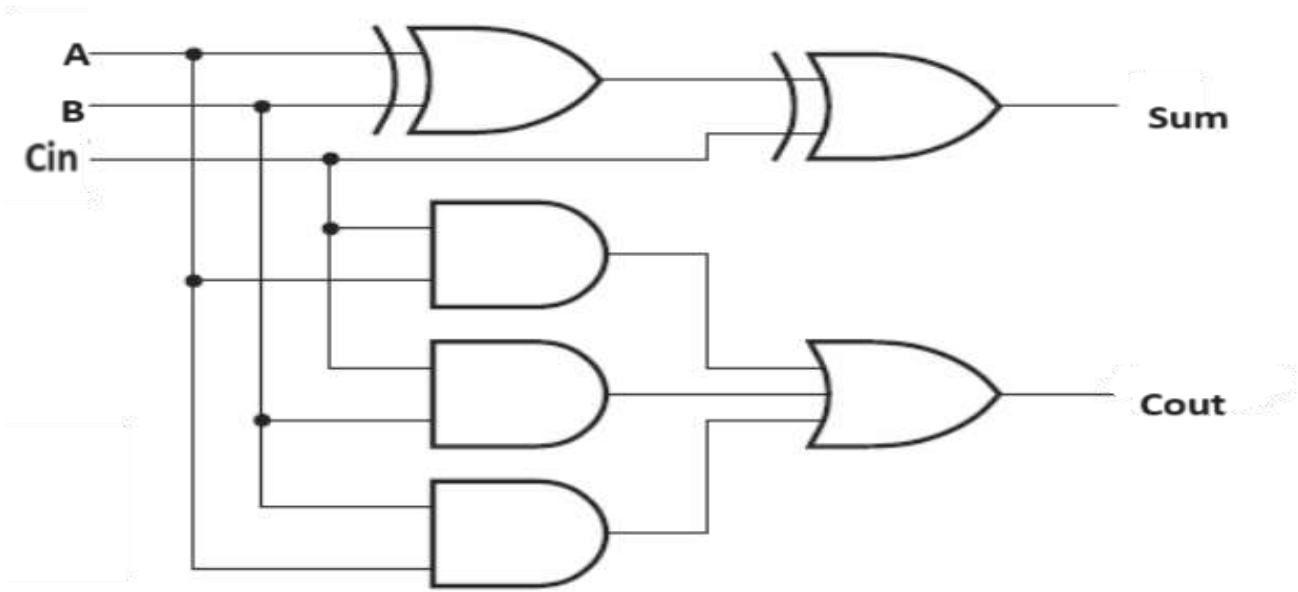
K-Map for C_{out}



K-Map for c_{out}

$$S = \bar{A}\bar{B}C_{in} + \bar{A}B\overline{C_{in}} + A\bar{B}\overline{C_{in}} + ABC_{in}$$
$$= C_{in}(\bar{A}\bar{B} + AB) + \overline{C_{in}}(\bar{A}B + A\bar{B})$$
$$= C_{in}\overline{(A \oplus B)} + \overline{C_{in}}(A \oplus B)$$
$$= (A \oplus B \oplus C_{in})$$

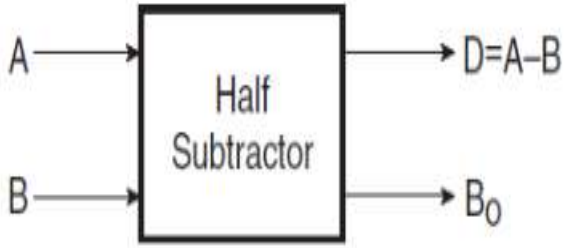
$$C_{out} = AB + BC_{in} + C_{in}A$$



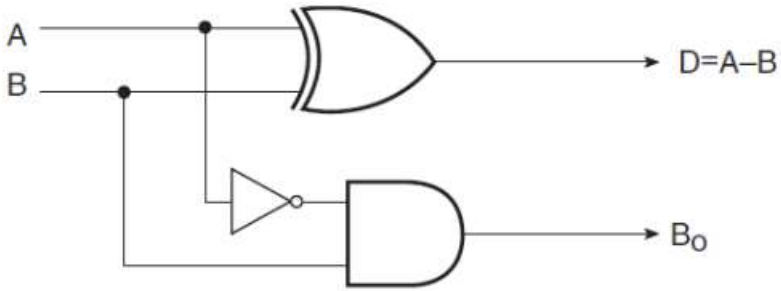
Half Subtractor

It produces the difference between the two binary bits at the input and produces two outputs difference and borrow (to indicate if a 1 has been borrowed)

In the subtraction (A), A is called a **Minuend bit** and B is called a **Subtrahend bit.**

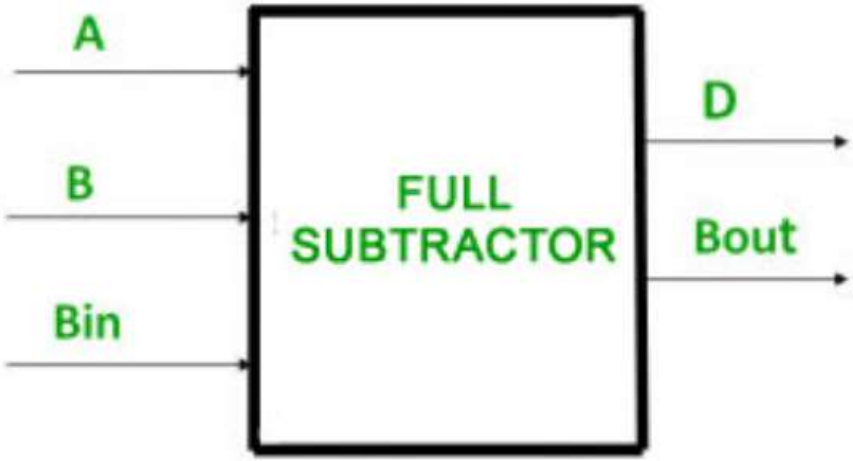


A	B	D	B_O
0	0	0	0
0	1	1	1
1	0	1	0
1	1	0	0



Full Subtractor

- A full subtractor is a **combinational circuit** that performs subtraction of two bits, one is minuend and other is subtrahend, taking into account borrow of the previous adjacent lower minuend bit. This circuit **has three inputs and two outputs**.
- The three inputs A, B and Bin, denote the minuend, subtrahend, and previous borrow, respectively. The two outputs, D and Bout represent the difference and output borrow, respectively.



INPUT			OUTPUT	
A	B	Bin	D	Bout
0	0	0	0	0
0	0	1	1	1
0	1	0	1	1
0	1	1	0	1
1	0	0	1	0
1	0	1	0	0
1	1	0	0	0
1	1	1	1	1

D

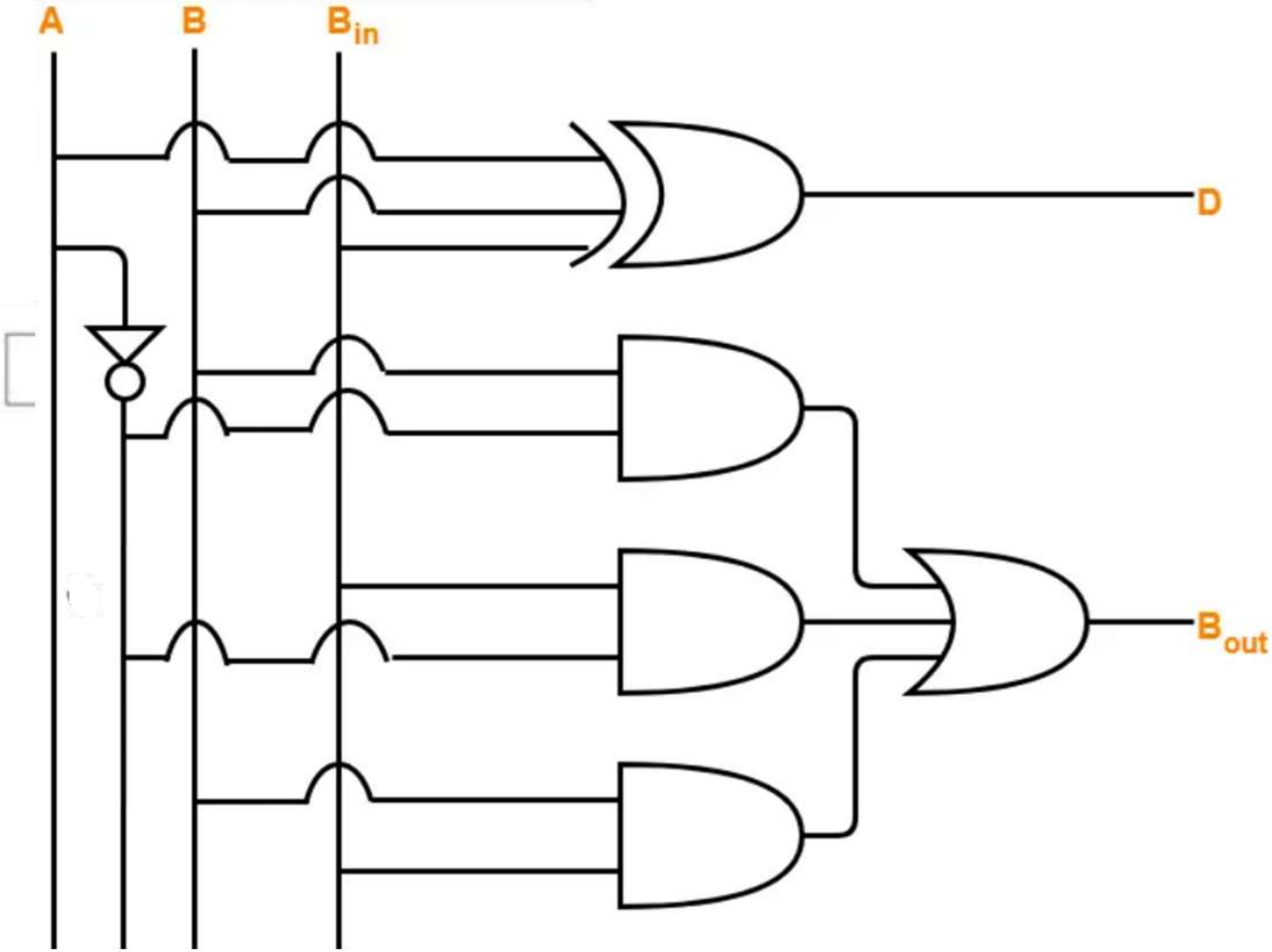
A \ B Bin	00	01	11	10
0	0	1	0	1
1	1	0	1	0

Bout

A \ B Bin	00	01	11	10
0	0	1	1	1
1	0	0	1	0

$$D = \bar{A}\bar{B}B_{in} + \bar{A}B\overline{B_{in}} + A\bar{B}\overline{B_{in}} + ABB_{in}$$
$$= B_{in}(\bar{A}\bar{B} + AB) + \overline{B_{in}}(\bar{A}B + A\bar{B})$$
$$= B_{in}\overline{(A \oplus B)} + \overline{B_{in}}(A \oplus B)$$
$$= (A \oplus B \oplus B_{in})$$

$$B_{out} = \bar{A}B_{in} + \bar{A}B + BB_{in}$$

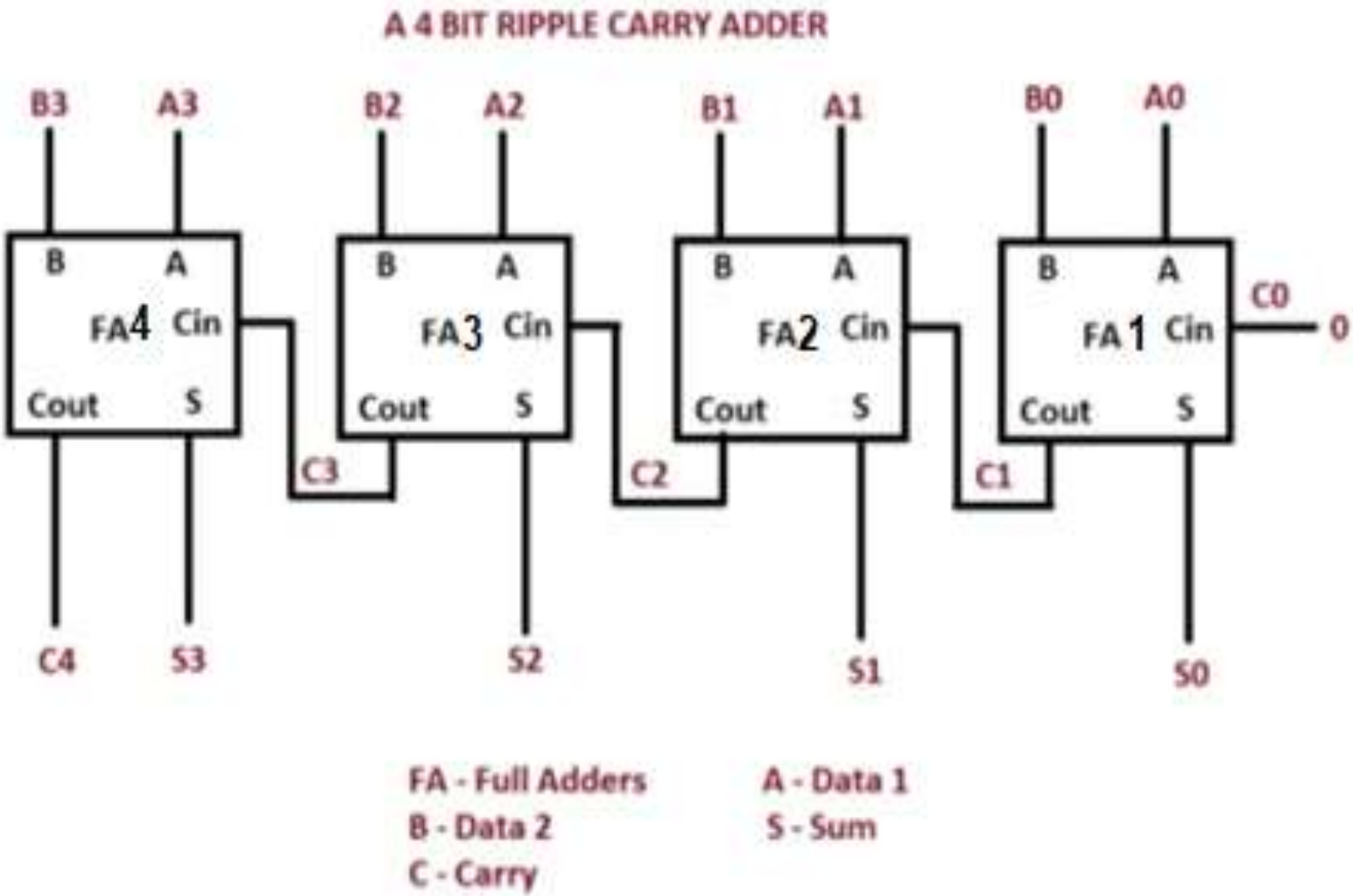

A
B
B_{in}
D
B_{out}

Ripple Carry Adder/ Parallel Adder

- Ripple Carry Adder is a combinational logic circuit.
- It is used to add two n-bit binary numbers.
- It requires n full adders in its circuit for adding two n-bit binary numbers.
- It is also known as **n-bit parallel adder**.

A 4-bit ripple carry adder is used to add two 4-bit binary numbers.

4-bit ripple carry adder/Parallel adder



Working of Ripple Carry Adder /Parallel adder

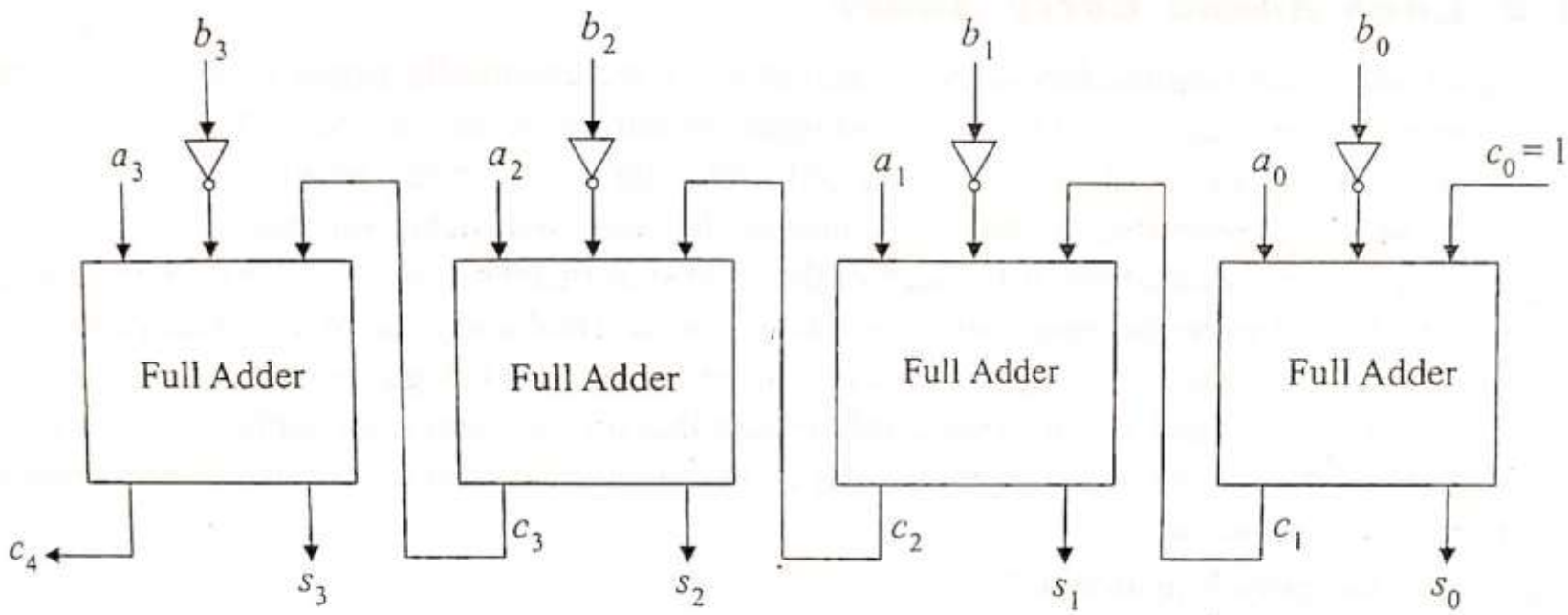
- Firstly the Full Adder FA1 adds A0 and B0 along with the carry C0 to generate the sum S0 (the first bit of the output sum) and the carry C1 which is connected to the next adder in chain.
- Next, the Full Adder FA2 uses this carry bit C1 to add with the input bits A1 and B1 to generate the sum S1(the second bit of the output sum) and the carry C2 which is again further connected to the next adder in chain and so on.
- The process continues till the last Full Adder FA4 uses the carry bit C2 to add with its input A3 and B3 to generate the last bit of the output S3 along with last carry bit C4.

Working of 4-bit Ripple Carry Adder with example

- Let's take an example of two input sequences 0101 and 1010. These are representing the A3 A2 A1 A0 and B3 B2 B1 B0 .
- As per this adder concept, input carry is 0.
- When Ao & Bo are applied at 1st full adder along with input carry 0.
- Here A0 =1 ; B0=0 ; C0=0
- Sum (S0) and carry (C1) will be generated as per the Sum and Carry equations of this adder. As per its theory, the output equation for the Sum = $A0 \oplus B0 \oplus C0$ and Carry = $A0B0 \oplus B0C0 \oplus C0A0$
- As per this equation, for 1st full adder S0 =1 and Carry output i.e., C1=0.
- Same like for next input bits A1 and B1, output S1 = 1 and C2 = 0. Here the important point is the second stage full adder gets input carry i.e., C1 which is the output carry of initial stage full adder.
- Like this will get the final output sequence (S3 S2 S1S0) = (1 1 1 1) and Output carry C4 = 0
- This is the addition process for 4-bit input sequences when it's applied to this ripple carry adder.

```
A3 A2 A1 A0 □   0101
B3 B2 B1 B0 □   1010
                C0□  0
-----------------------------
 S3S2S1S0□ 1111
C4□  0
```

Parallel Subtractor



Parallel adder performing parallel subtraction

Working of Parallel Subtractor

- The parallel binary subtractor is formed by combination of all full adders with subtrahend complement input.
- This operation considers that the addition of minuend along with the 2's complement of the subtrahend is equal to their subtraction.
- Firstly the 1's complement of B is obtained by the NOT gate and 1 can be added through the carry to find out the 2's complement of B. This is further added to A to carry out the arithmetic subtraction.
- The process continues till the last full adder uses the carry bit to add with its input A and 2's complement of B to generate the last bit of the output S3 along last carry bit C_4.

For Subtraction

For a>b, a=b

- If a carry is generated (C4=1) , then ignore the carry and result will be the difference between a and b.

For a<b

- If no carry (C4=0), then the difference is 2’s compliment of the result with negative sign

4-bit Parallel Adder/Subtractor

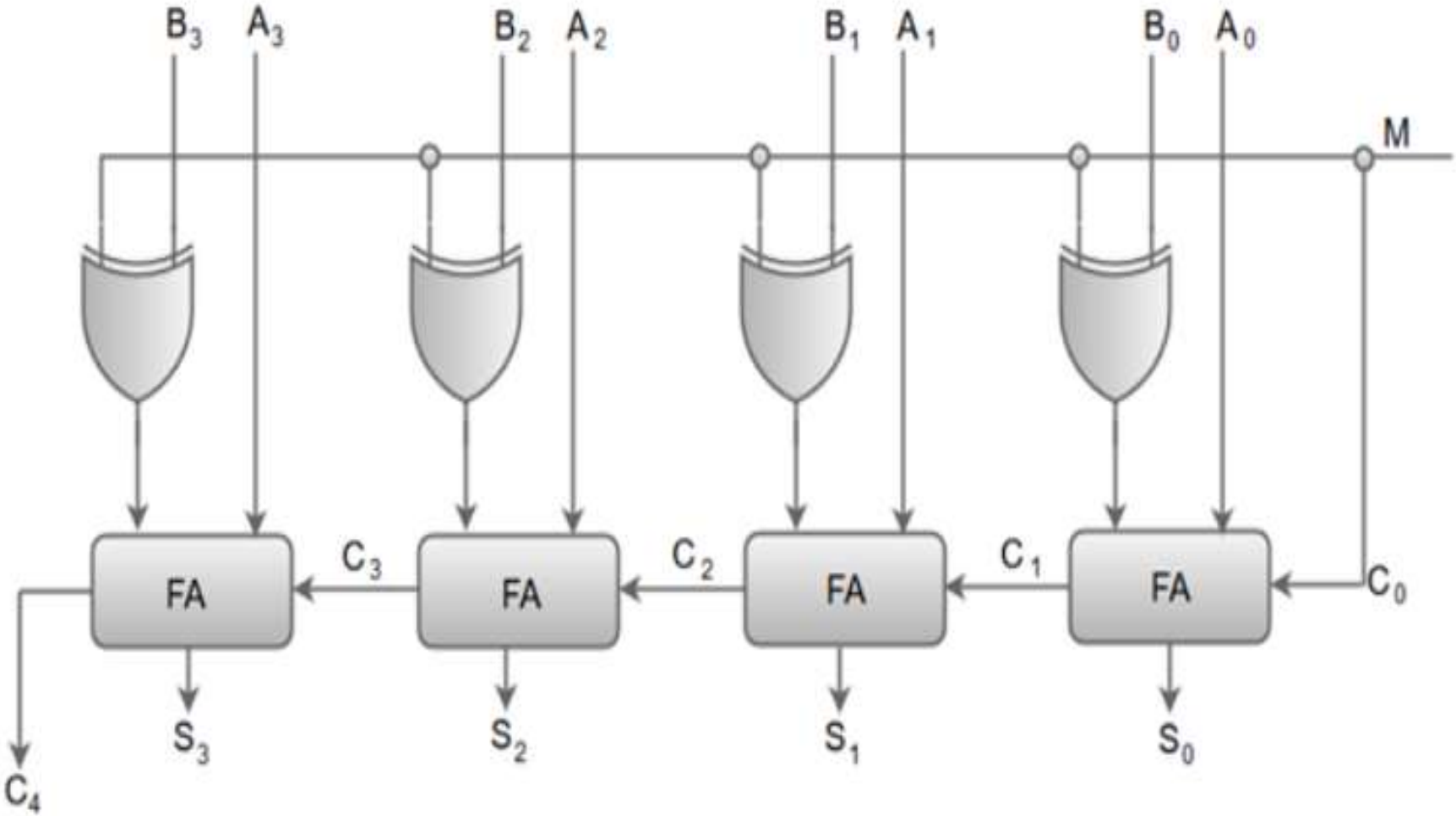
- In Digital Circuits, a **Binary adder/subtractor** is capable of both the addition and subtraction of binary numbers in one circuit itself.
- The operation is performed depending on the binary value the control signal holds. It is one of the components of the ALU (Arithmetic Logic Unit).
- Let's consider two 4-bit binary numbers A and B as inputs to the Digital Circuit for the operation with digits

```
A0 A1 A2 A3 for A

B0 B1 B2 B3 for B
```

The circuit consists of 4 full adders since we are performing operations on 4-bit numbers. There is a control line M that holds a binary value of either 0 or 1 which determines that the operation is carried out is addition or subtraction.

4 bit adder-subtractor:



- When the mode input (M) is at a low logic, i.e. '0', the circuit acts as an adder and when the mode input is at a high logic, i.e. '1', the circuit acts as a subtractor.
- The exclusive-OR gate connected in series receives input M and one of the inputs B.
- When M is at a low logic, we have $B \oplus 0 = B$. The full adders receive the value of B, the input carry is 0, and the circuit performs A plus B.
- When M is at a high logic, we have $B \oplus 1 = B'$ and C0 = 1. The B inputs are complemented, and a 1 is added through the input carry. The circuit performs the operation A plus the 2's complement of B.

Examples

```
M=0: A = 0001,B = 0000
S = 0001, C4= 0

M=0: A = 0010,B = 0100
S = 0110, C4 = 0

M=0: A = 1011,B = 0110
S = 0001, C4 = 1
M=1: A = 0001,B = 0000
S = 0001,C4= 1

M=1: A = 0010, B = 0100
S = 1110, C4= 0

M=1: A = 1011, B = 0110
S = 0101, C4 = 1

M=1: A = 0101, B = 0011
S = 0010, C4= 1
```

1-bit comparator

Truth table for a one-bit comparator

A	B	A EQ B	$A > B$	$A < B$
0	0	1	0	0
0	1	0	0	1
1	0	0	1	0
1	1	1	0	0

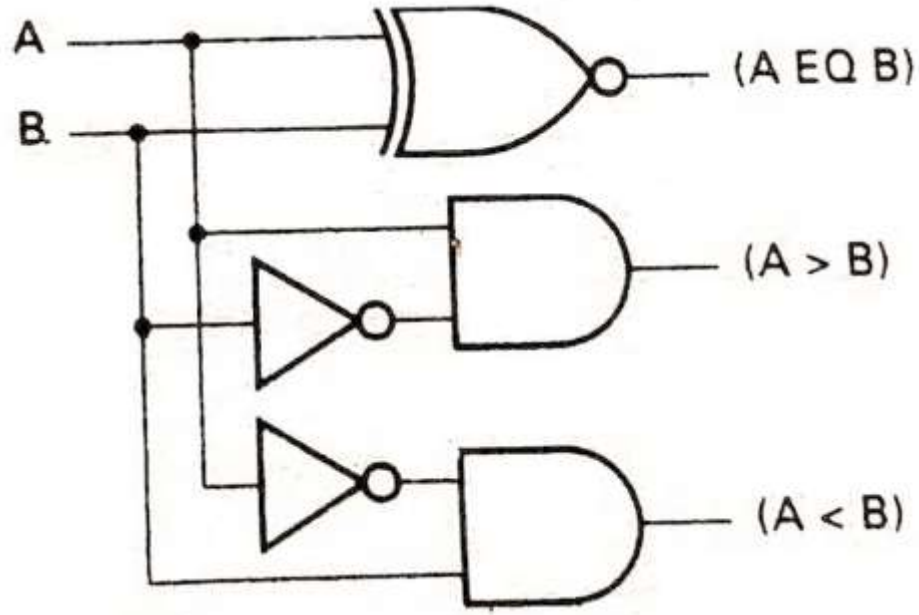
The equations for each output are

$$A \text{ EQ } B = A'B' + AB = (A \oplus B)'$$

$$A > B = AB'$$

$$A < B = A'B$$

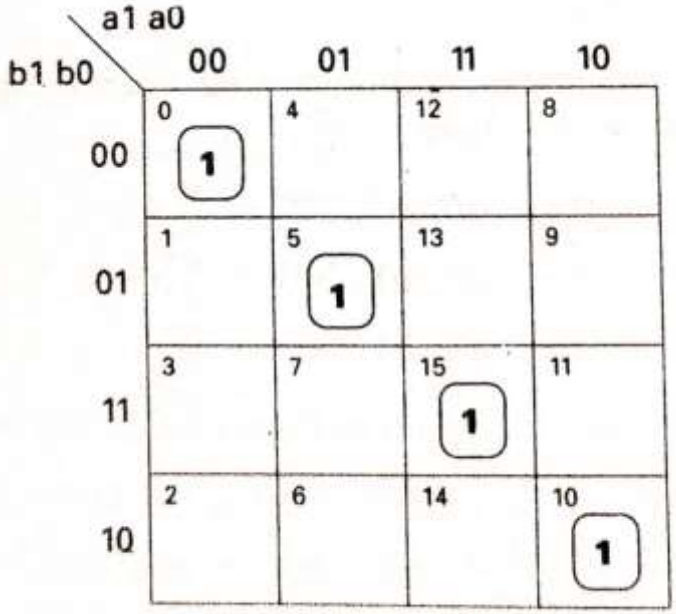
Logic for a single-bit comparator



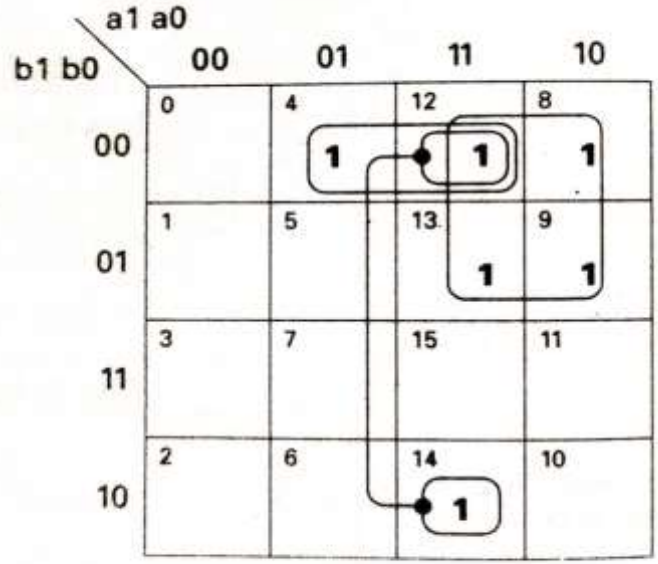
2- bit comparator

Truth table for a two-bit magnitude comparator

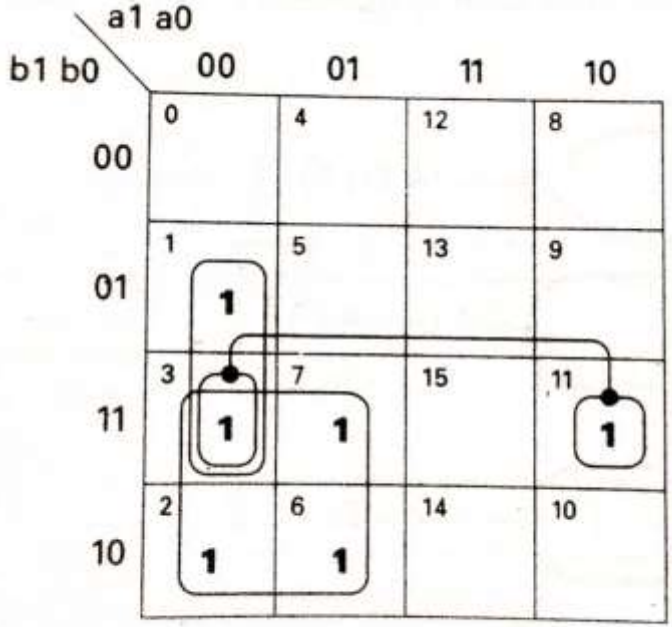
Inputs				Outputs		
A		B				
$a1$	$a0$	$b1$	$b0$	$A = B$	$A > B$	$A < B$
0	0	0	0	1	0	0
0	0	0	1	0	0	1
0	0	1	0	0	0	1
0	0	1	1	0	0	1
0	1	0	0	0	1	0
0	1	0	1	1	0	0
0	1	1	0	0	0	1
0	1	1	1	0	0	1
1	0	0	0	0	1	0
1	0	0	1	0	1	0
1	0	1	0	1	0	0
1	0	1	1	0	0	1
1	1	0	0	0	1	0
1	1	0	1	0	1	0
1	1	1	0	0	1	0
1	1	1	1	1	0	0



(a) (A EQ B)



(b) (A > B)



(c) (A < B)

$$(A \text{ EQ } B) = f(a1, a0, b1, b0) = \sum(0, 5, 10, 15)$$

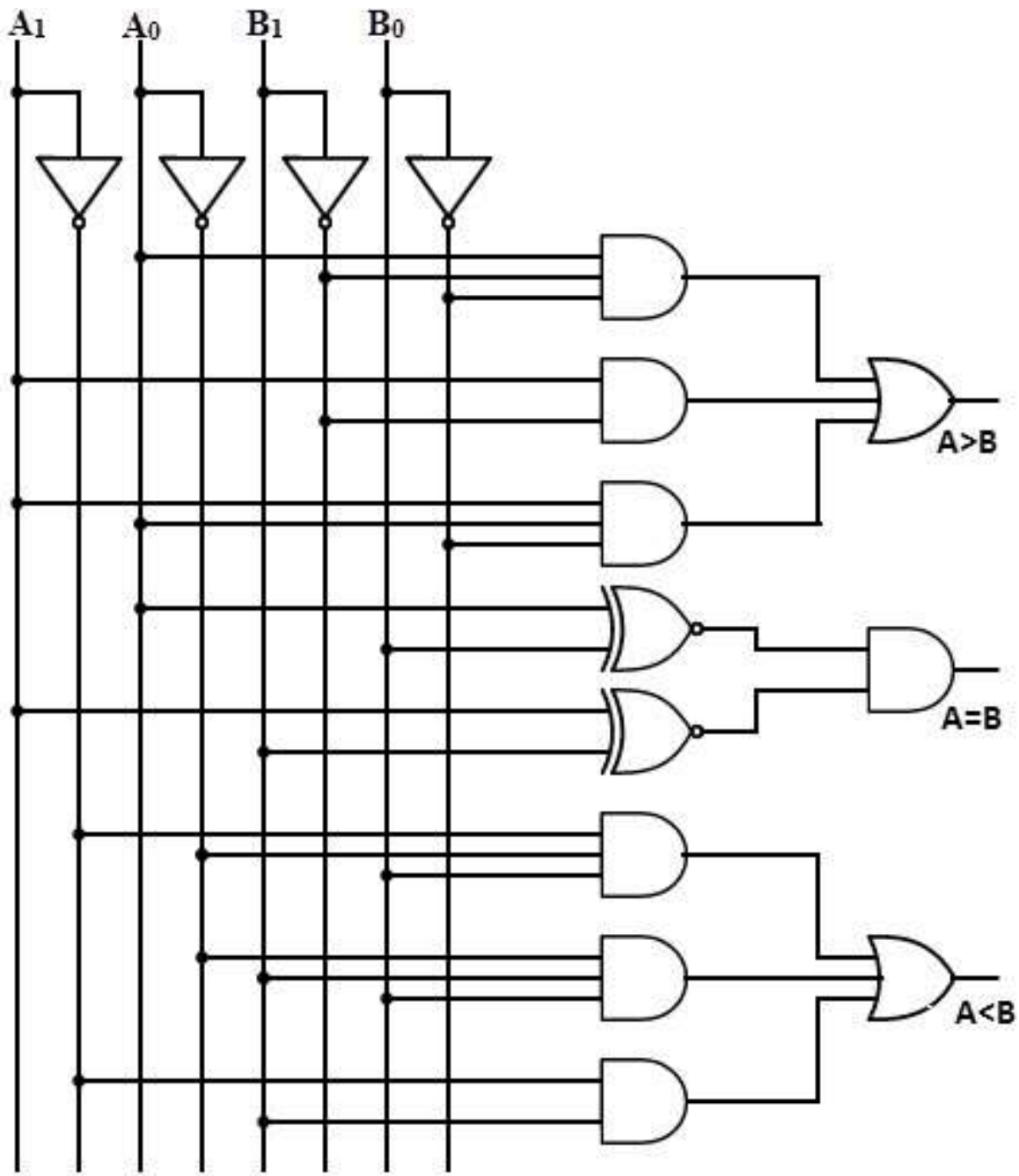
$$(A > B) = f(a1, a0, b1, b0) = \sum(4, 8, 9, 12, 13, 14)$$

$$(A < B) = f(a1, a0, b1, b0) = \sum(1, 2, 3, 6, 7, 11)$$

$$(A \text{ EQ } B) = a1'a0'b1'b0' + a1'a0b1'b0 + a1a0b1b0 + a1a0'b1b0'$$

$$(A > B) = a1b1' + a0b1'b0' + a1a0b0'$$

$$(A < B) = a1'b1 + a1'a0'b0 + a0'b1b0$$


A1
A0
B1
B0
A>B
A=B
A<B

Multiplexer(MUX)

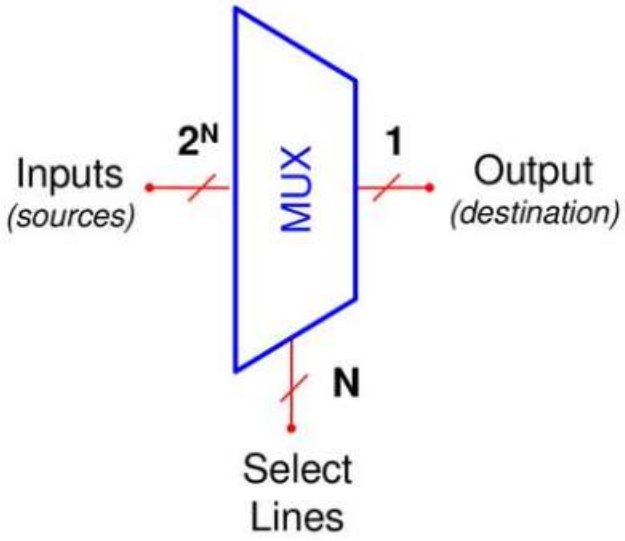
A MUX is a digital switch that has multiple inputs (sources) and a single output (destination).

The select lines determine which input is connected to the output.

MUX Types

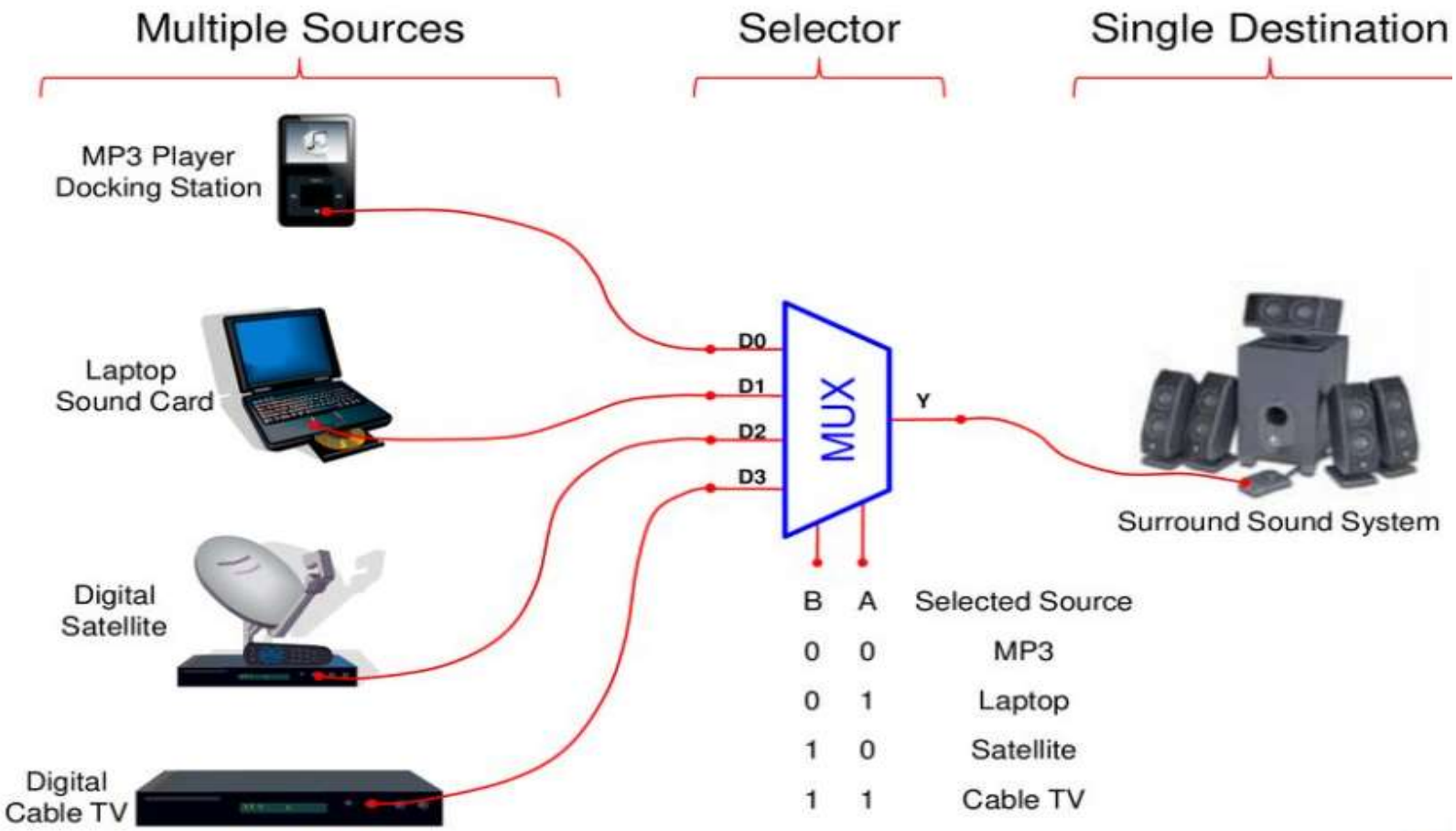
→ 2-to-1 (1 select line)
→ 4-to-1 (2 select lines)
→ 8-to-1 (3 select lines)
→ 16-to-1 (4 select lines)

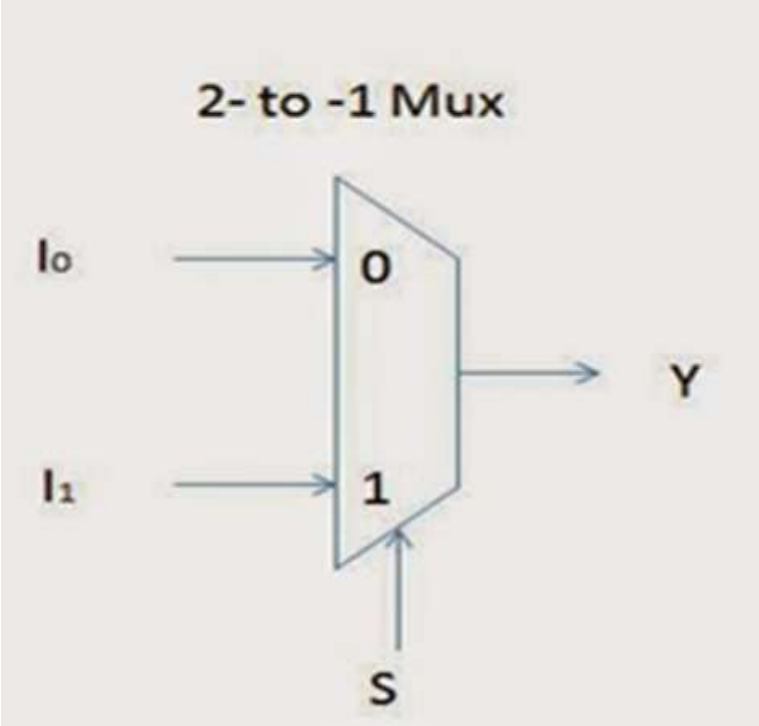
Multiplexer
Block Diagram



- Also called as Data Selector.
- Multiplex means *Many into one*.

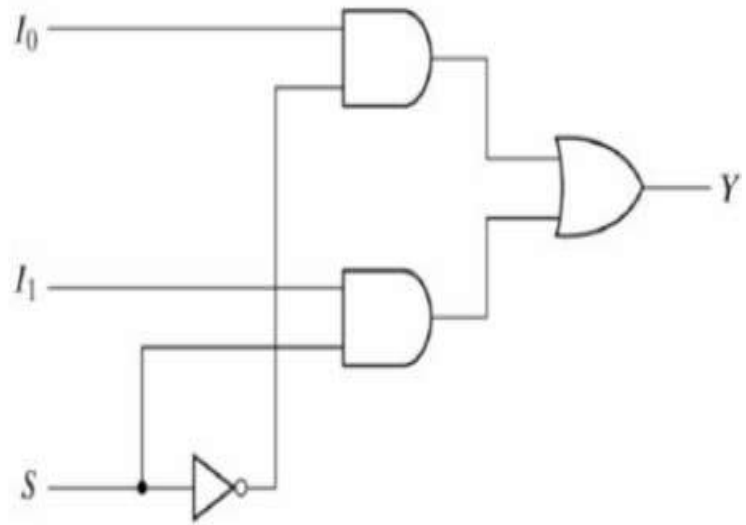
Application of MUX

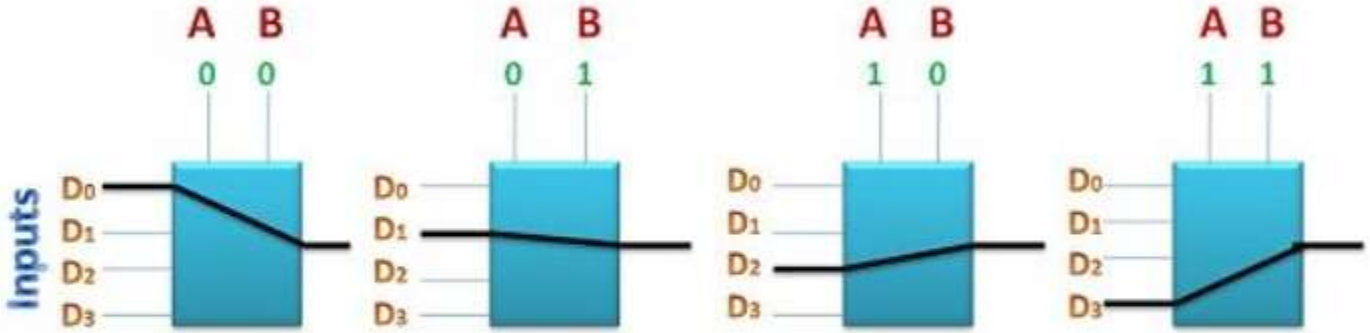




S	Y
0	I_0
1	I_1

$$Y = \bar{S} I_0 + S\, I_1$$

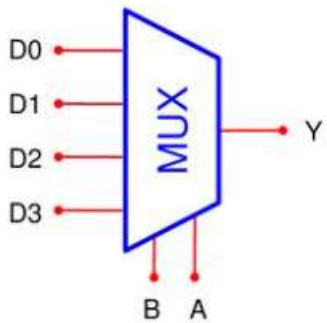


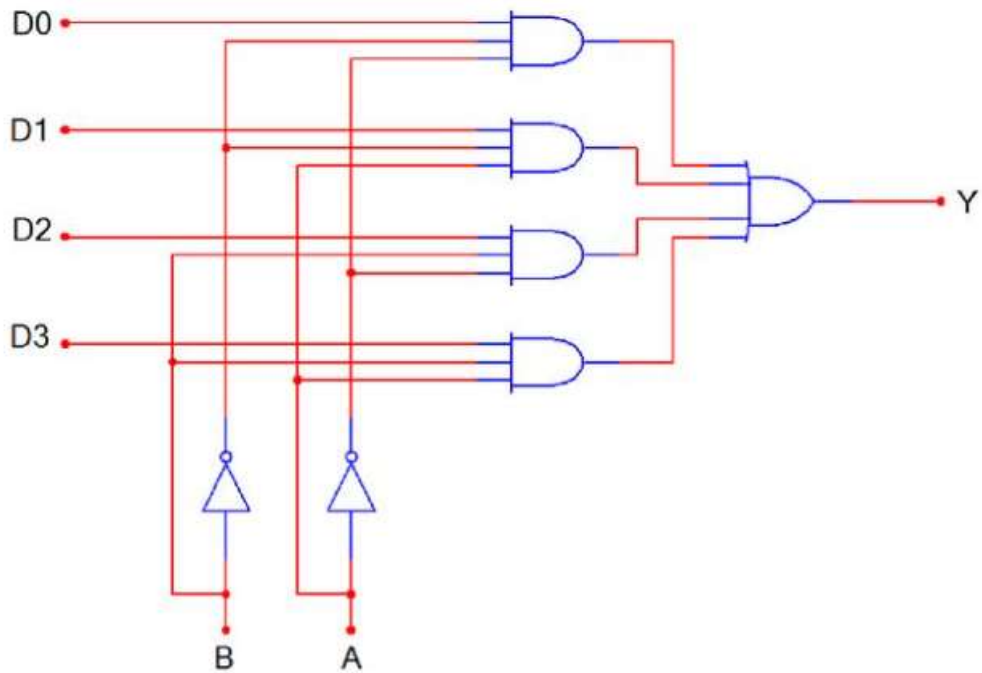


Input Line Selection by MUX

Electronics Coach

4:1 Multiplexer

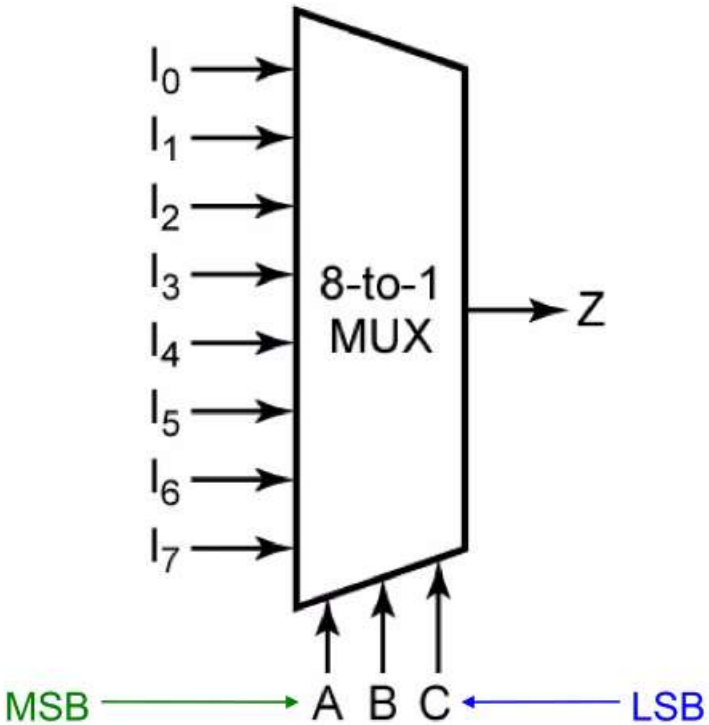




B	A	Y
0	0	D0
0	1	D1
1	0	D2
1	1	D3

$$Y = \bar{B}\bar{A}D_0 + \bar{B}AD_1 + B\bar{A}D_2 + BAD_3$$

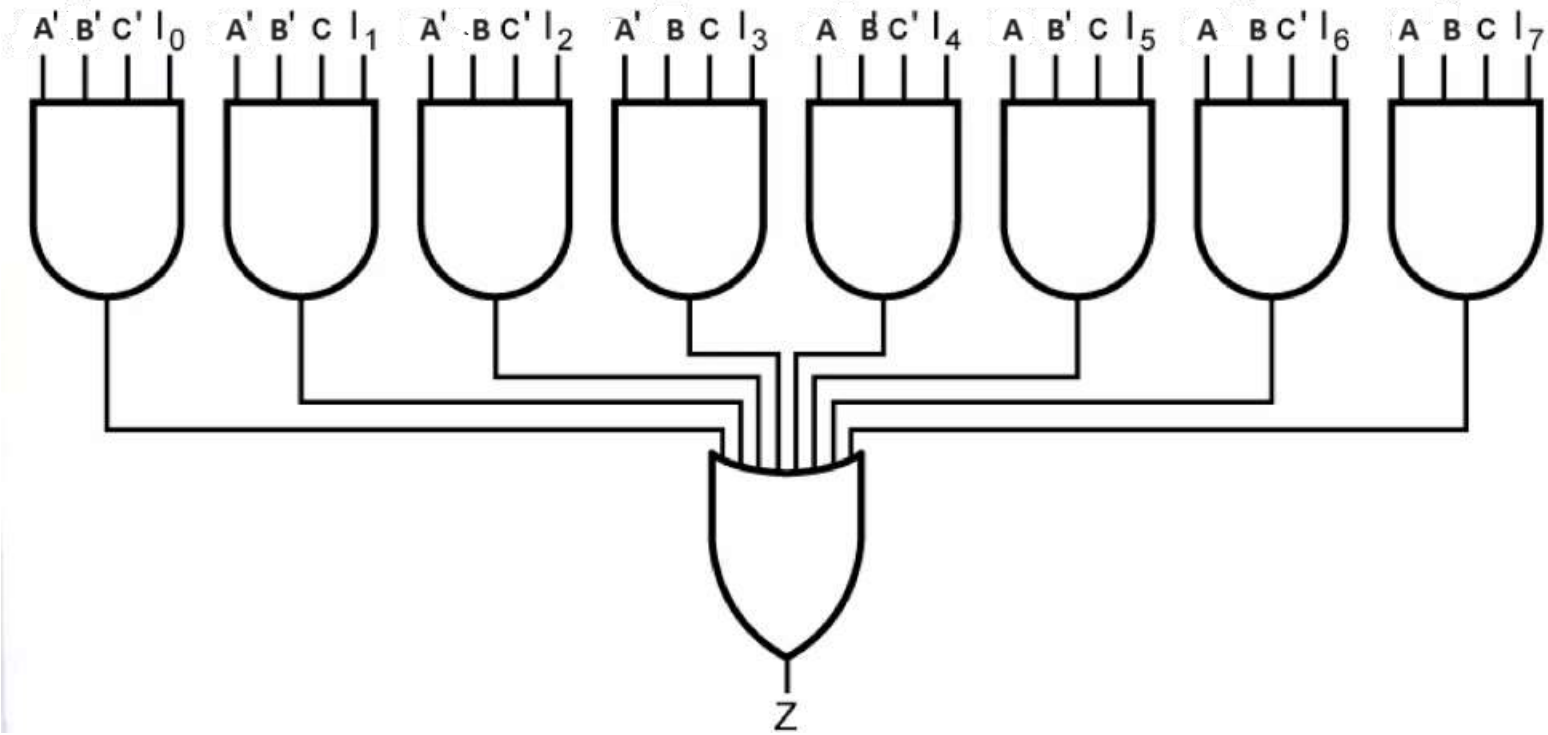
8:1 Multiplexer



A	B	C	F
0	0	0	I_0
0	0	1	I_1
0	1	0	I_2
0	1	1	I_3
1	0	0	I_4
1	0	1	I_5
1	1	0	I_6
1	1	1	I_7

$$Z = A'.B'.C'.I_0 + A'.B'.C.I_1 + A'.B.C'.I_2 + A'.B.C.I_3 + A.B'.C'.I_4 + A.B'.C.I_5 + A'.B.C'.I_6 + A.B.C.I_7$$

8:1 Multiplexer



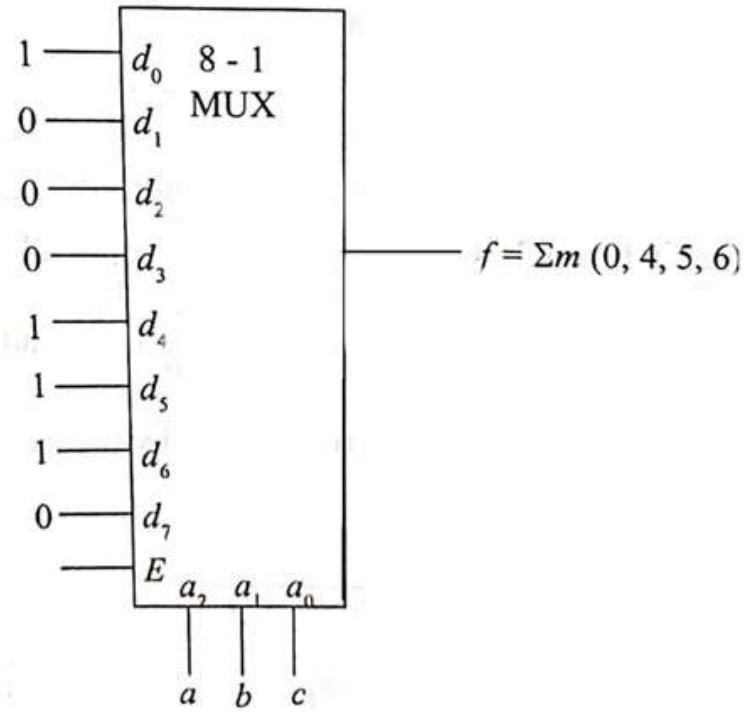
Implement the following using a 8 to 1 MUX

F(a,b,c) =$\sum m(0,4,5,6)$

Solution: Let $d_0 = d_4 = d_5 = d_6 = 1$ and

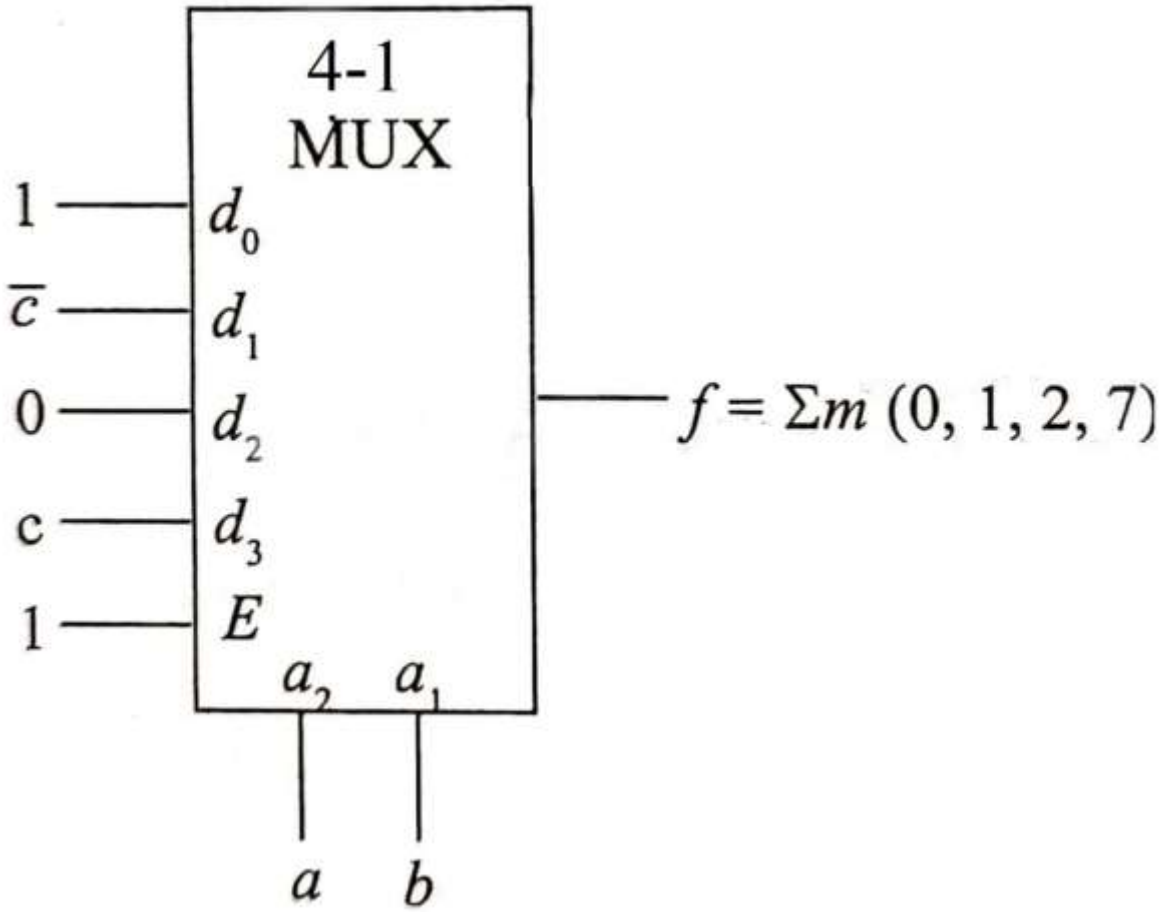
$d_1 = d_2 = d_3 = d_7 = 0$ in a 8 to 1 MUX.

The arrangement is shown below.



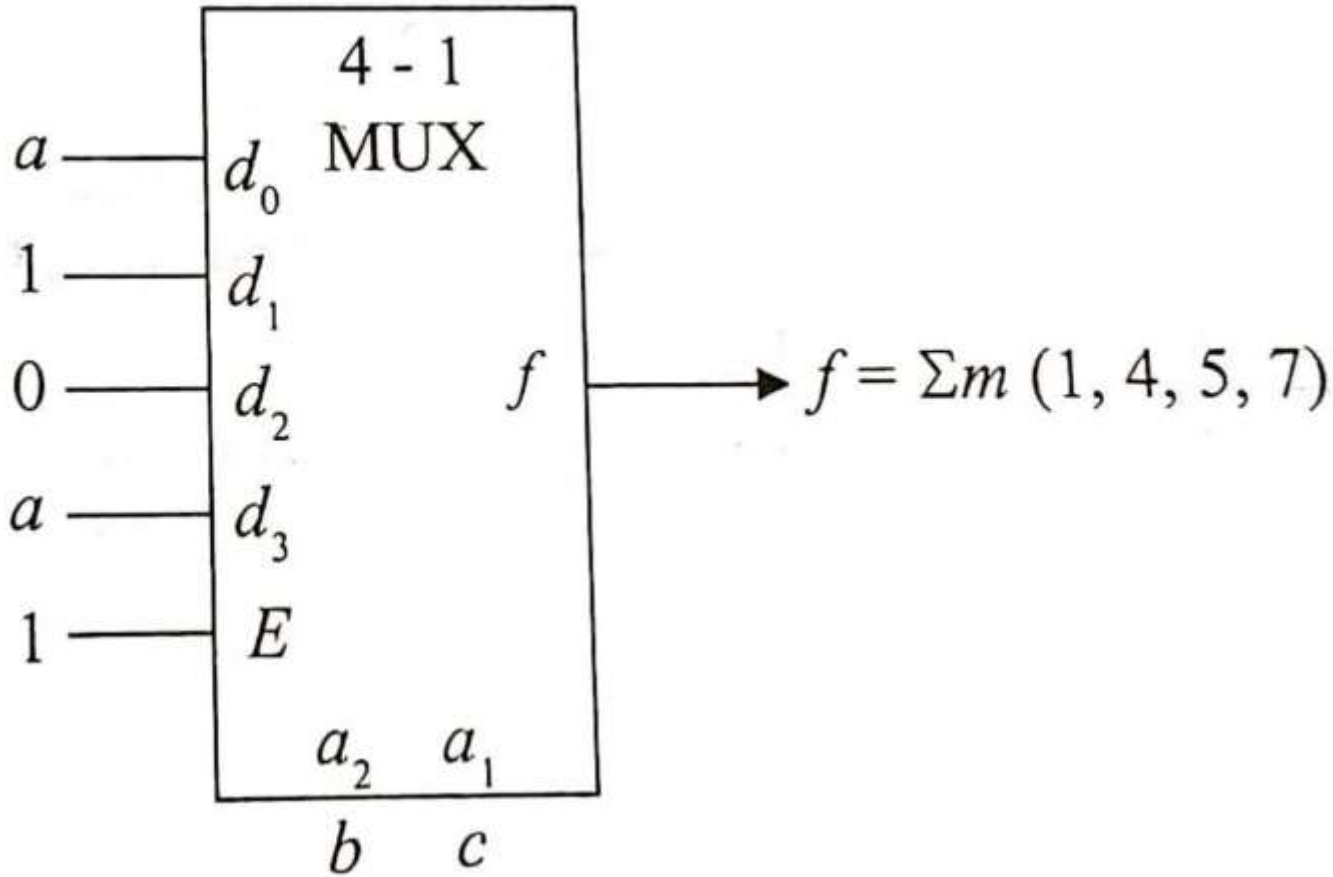
Implement the following using a 4 to 1 MUX

f(a,b,c) =$\sum m(0,1,2,7)$



Implement the following using a 4 to 1 MUX

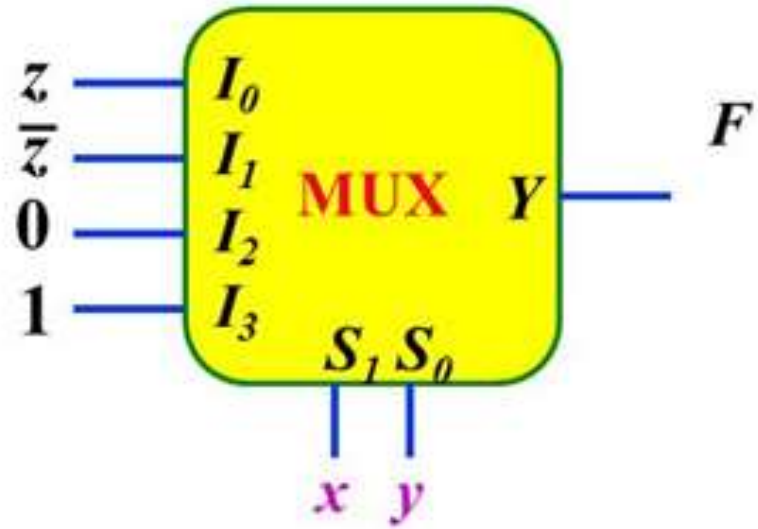
f(a,b,c) =$\sum m(1,4,5,7)$



★ Example

$F(x, y, z) = \sum(1, 2, 6, 7)$

x	y	z	F	
0	0	0	0	$F = z$
0	0	1	1	
0	1	0	1	$F = \overline{z}$
0	1	1	0	
1	0	0	0	$F = 0$
1	0	1	0	
1	1	0	1	$F = 1$
1	1	1	1	

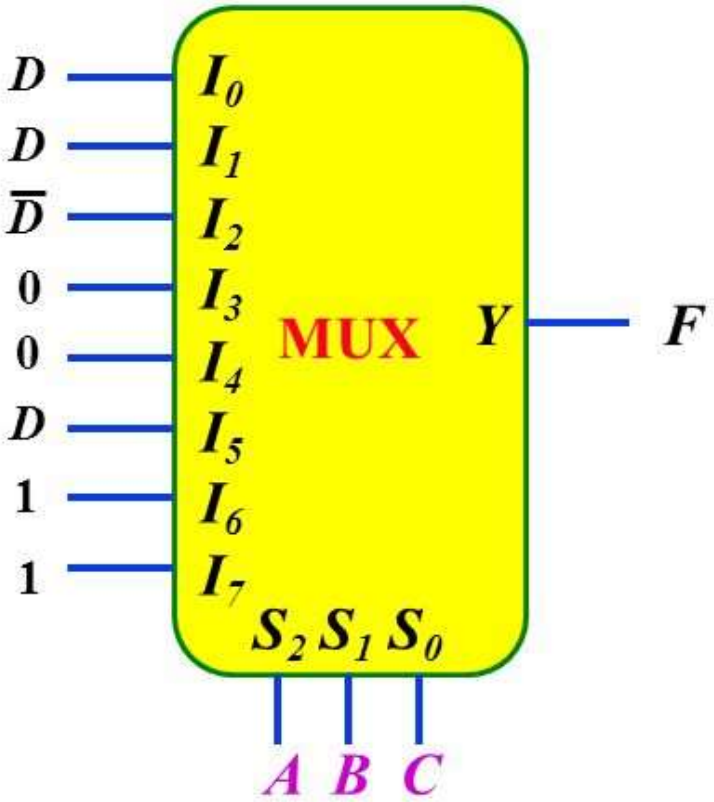


Implementation Using Multiplexers

★ **Example**

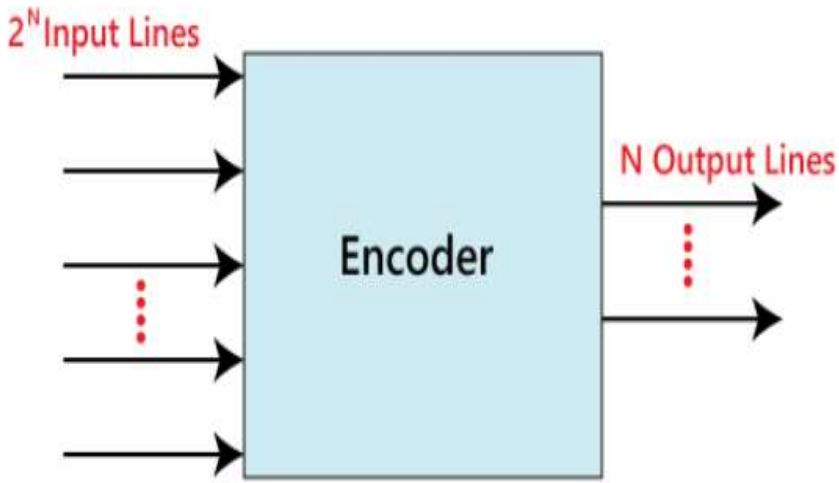
$$F(A, B, C, D) = \sum(1, 3, 4, 11, 12, 13, 14, 15)$$

A	B	C	D	F	
0	0	0	0	0	$F = D$
0	0	0	1	1	
0	0	1	0	0	$F = D$
0	0	1	1	1	
0	1	0	0	1	$F = \overline{D}$
0	1	0	1	0	
0	1	1	0	0	$F = 0$
0	1	1	1	0	
1	0	0	0	0	$F = 0$
1	0	0	1	0	
1	0	1	0	0	$F = D$
1	0	1	1	1	
1	1	0	0	1	$F = 1$
1	1	0	1	1	
1	1	1	0	1	$F = 1$
1	1	1	1	1	



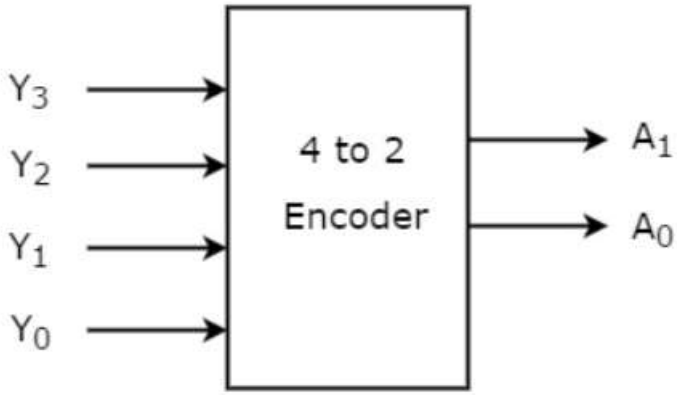
Encoders

Combinational circuits that change the binary information into N output lines.
The binary information is passed in the form of 2^N input lines.
The output lines define the N-bit code for the binary information.



4 to 2 Encoder

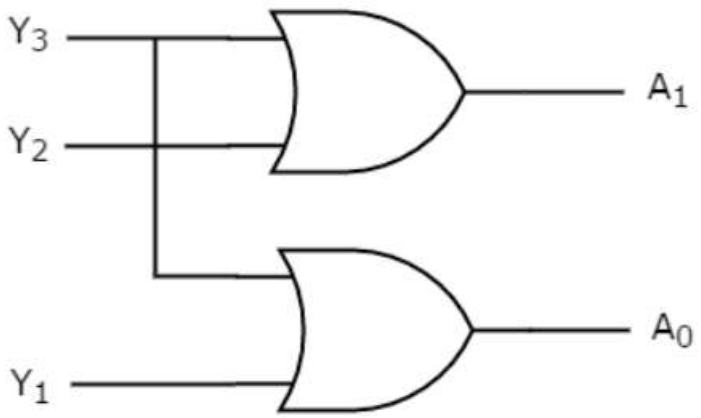
Let 4 to 2 Encoder has four inputs Y_3, Y_2, Y_1 & Y_0 and two outputs A_1 & A_0. The **block diagram** of 4 to 2 Encoder



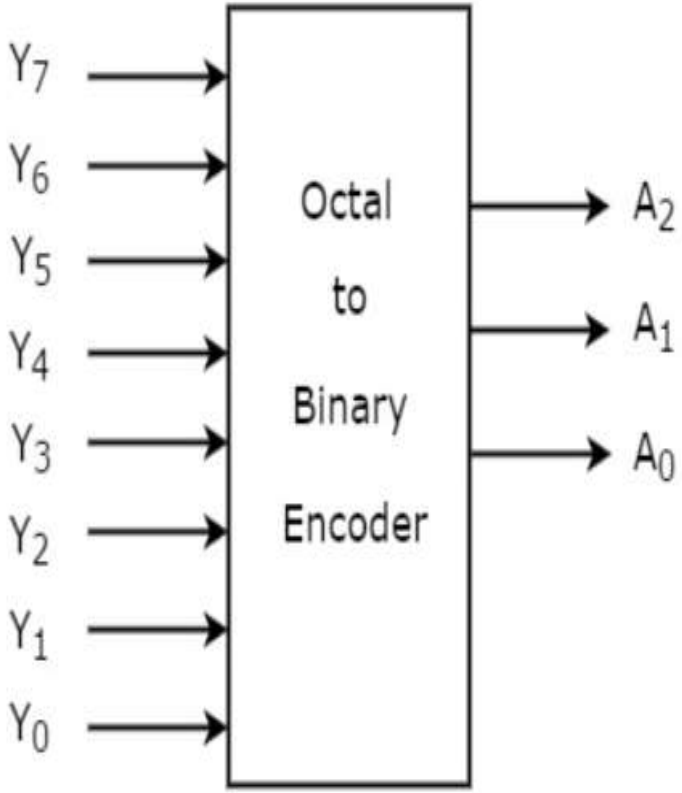
Inputs				Outputs	
Y_3	Y_2	Y_1	Y_0	A_1	A_0
0	0	0	1	0	0
0	0	1	0	0	1
0	1	0	0	1	0
1	0	0	0	1	1

$$A_1 = Y_3 + Y_2$$

$$A_0 = Y_3 + Y_1$$


Y_3
Y_2
Y_1
A_1
A_0

Octal to Binary(8 to 3 line) Encoder

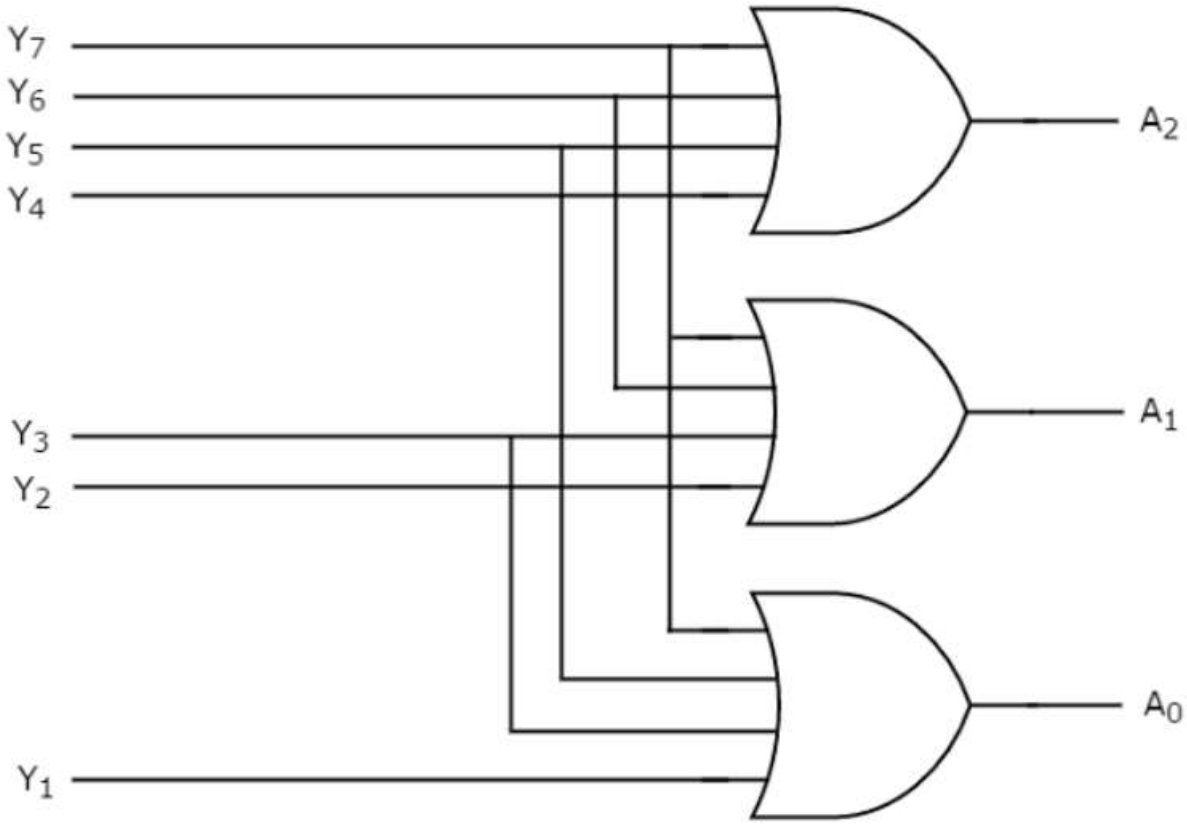


Inputs								Outputs		
Y_7	Y_6	Y_5	Y_4	Y_3	Y_2	Y_1	Y_0	A_2	A_1	A_0
0	0	0	0	0	0	0	1	0	0	0
0	0	0	0	0	0	1	0	0	0	1
0	0	0	0	0	1	0	0	0	1	0
0	0	0	0	1	0	0	0	0	1	1
0	0	0	1	0	0	0	0	1	0	0
0	0	1	0	0	0	0	0	1	0	1
0	1	0	0	0	0	0	0	1	1	0
1	0	0	0	0	0	0	0	1	1	1

$$A_2 = Y_7 + Y_6 + Y_5 + Y_4$$

$$A_1 = Y_7 + Y_6 + Y_3 + Y_2$$

$$A_0 = Y_7 + Y_5 + Y_3 + Y_1$$


Y_7
Y_6
Y_5
Y_4
Y_3
Y_2
Y_1
A_2
A_1
A_0

Drawbacks of Encoder

- There is an ambiguity, when all outputs of encoder are equal to zero. Because, it could be the code corresponding to the inputs, when only least significant input is one or when all inputs are zero.

- If more than one input is active High, then the encoder produces an output, which may not be the correct code. For **example**, if both Y_3 and Y_6 are '1', then the encoder produces 111 at the output. This is neither equivalent code corresponding to Y_3, when it is '1' nor the equivalent code corresponding to Y_6, when it is '1'.

Decoders

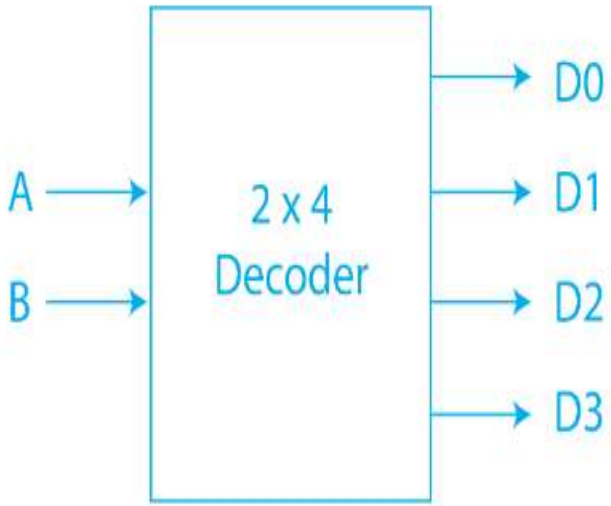
A 2 to 4 decoder is a combinational logic circuit that takes two input lines, typically labeled A and B, and generates four output lines, usually labeled Y0, Y1, Y2, and Y3.

The decoder analyses the input combination and activates the corresponding output line. Each output line represents a unique combination of the input lines.

Combinational circuit that has ‘n’ input lines and a maximum of 2^n output lines.

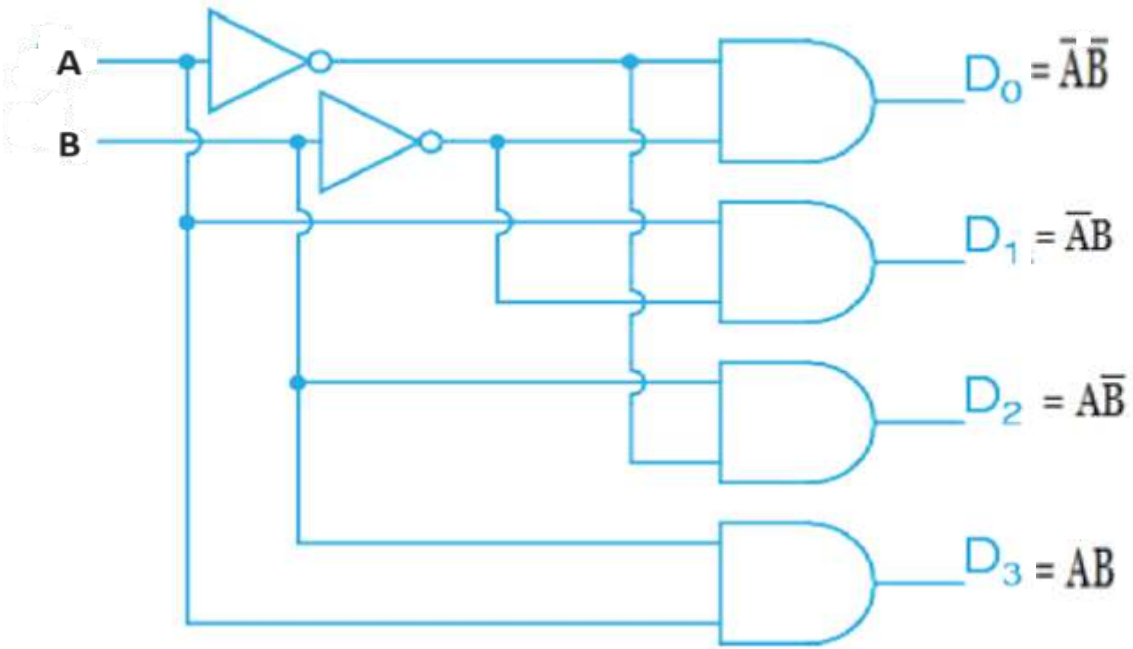
The outputs of the decoder are nothing but the min terms of ‘n’ input variables lines, when it is enabled

Decoders

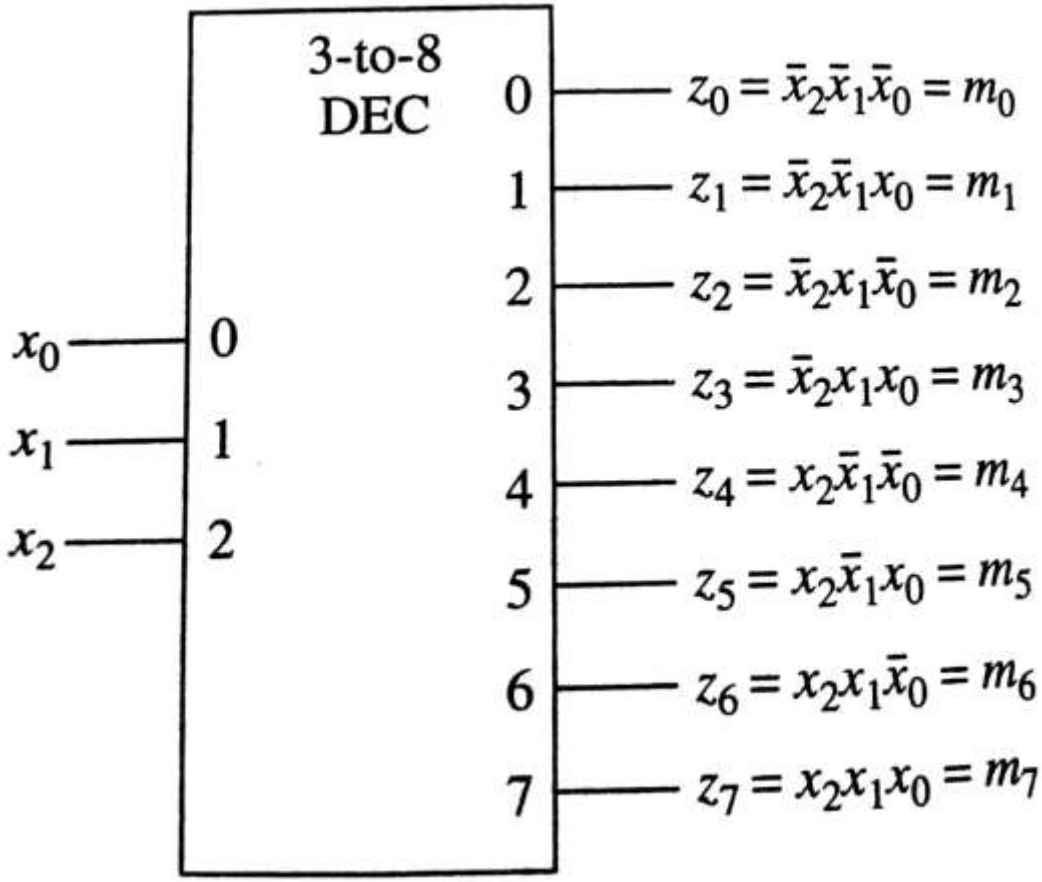


A	B	Y0	Y1	Y2	Y3
0	0	1	0	0	0
0	1	0	1	0	0
1	0	0	0	1	0
1	1	0	0	0	1

Decoders

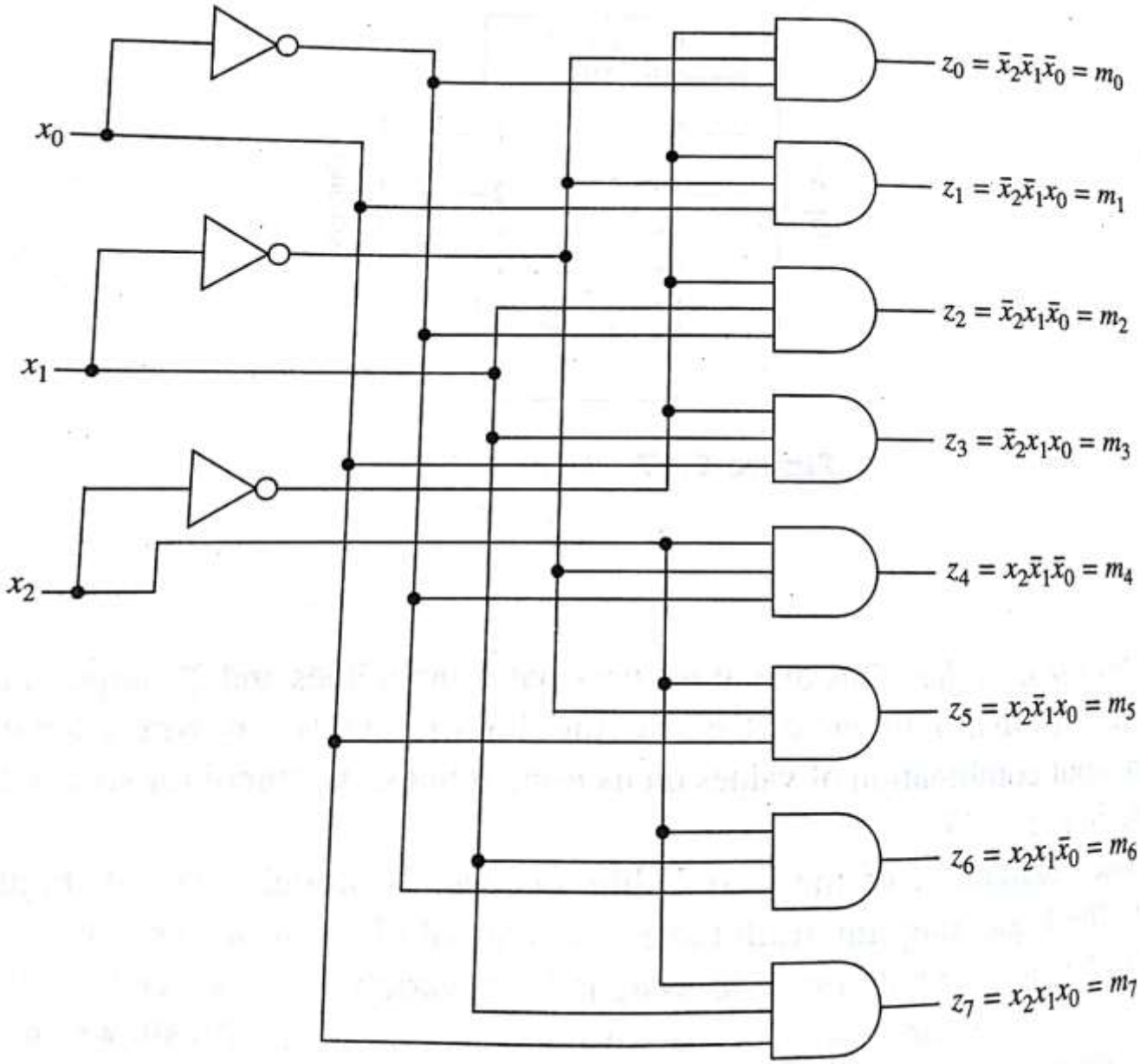


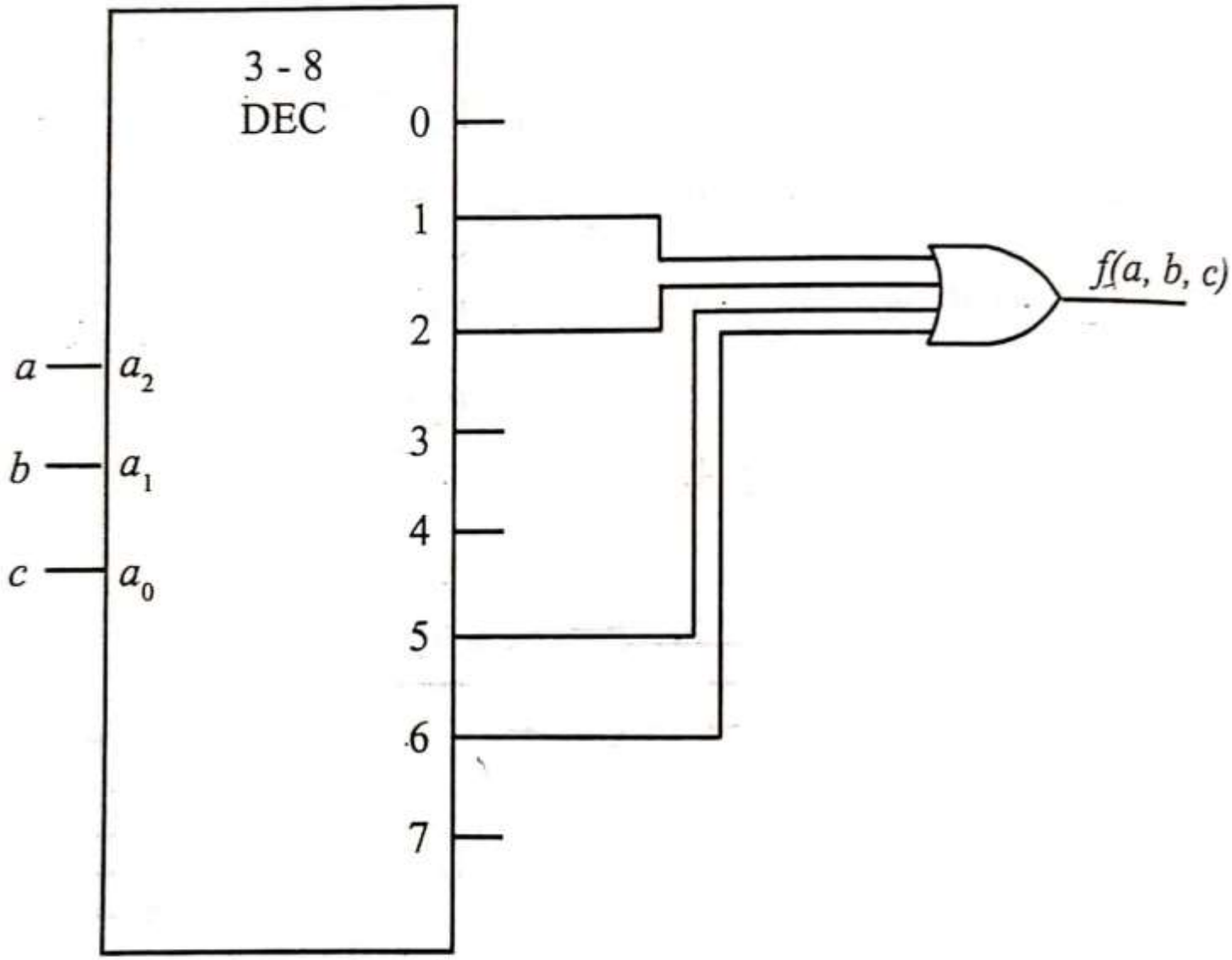
3 to 8 decoder



Truth Table of 3:8 decoder

Inputs			Outputs							
x_2	x_1	x_0	z_0	z_1	z_2	z_3	z_4	z_5	z_6	z_7
0	0	0	1	0	0	0	0	0	0	0
0	0	1	0	1	0	0	0	0	0	0
0	1	0	0	0	1	0	0	0	0	0
0	1	1	0	0	0	1	0	0	0	0
1	0	0	0	0	0	0	1	0	0	0
1	0	1	0	0	0	0	0	1	0	0
1	1	0	0	0	0	0	0	0	1	0
1	1	1	0	0	0	0	0	0	0	1


x_0
x_1
x_2
$z_0 = \bar{x}_2\bar{x}_1\bar{x}_0 = m_0$
$z_1 = \bar{x}_2\bar{x}_1x_0 = m_1$
$z_2 = \bar{x}_2x_1\bar{x}_0 = m_2$
$z_3 = \bar{x}_2x_1x_0 = m_3$
$z_4 = x_2\bar{x}_1\bar{x}_0 = m_4$
$z_5 = x_2\bar{x}_1x_0 = m_5$
$z_6 = x_2x_1\bar{x}_0 = m_6$
$z_7 = x_2x_1x_0 = m_7$

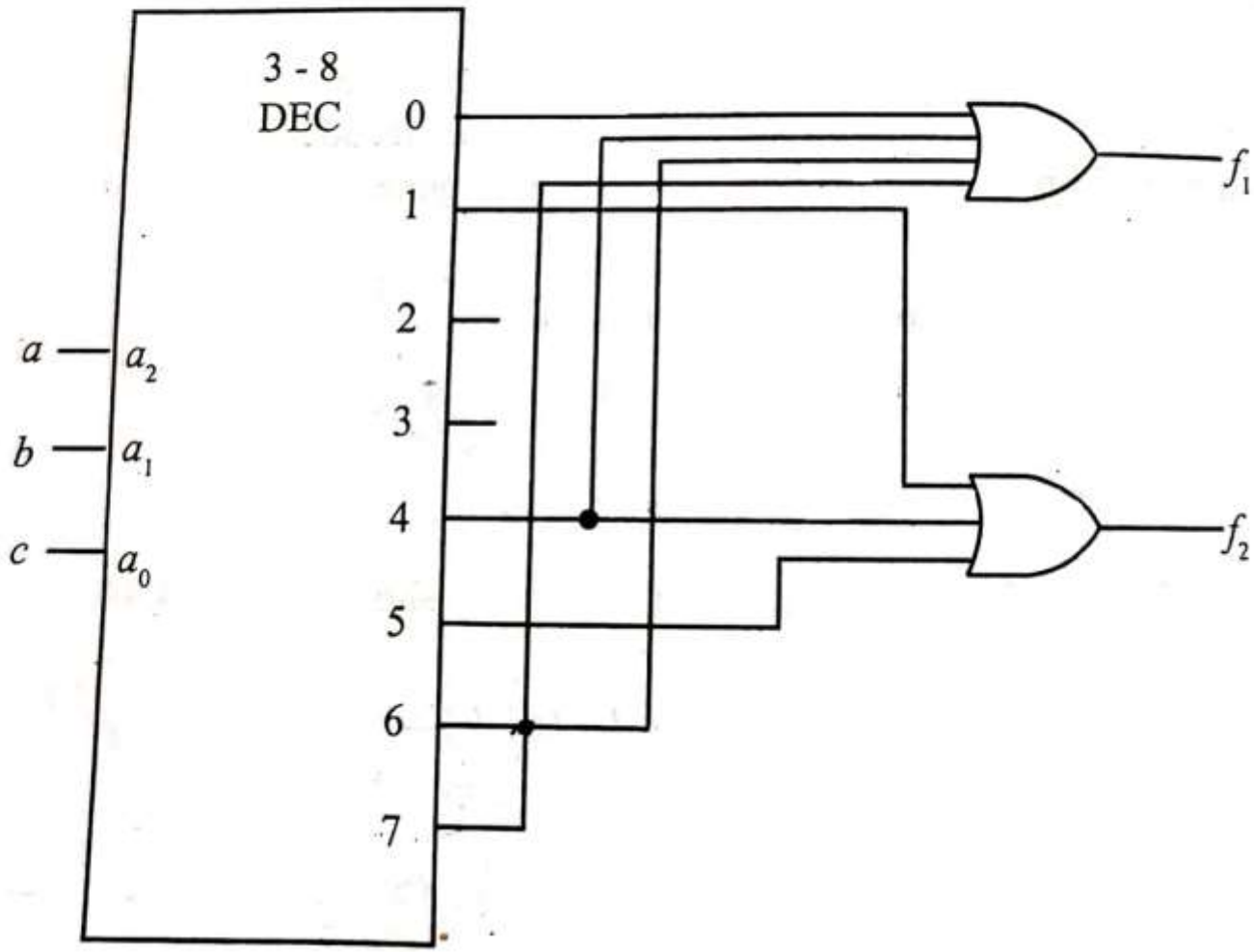


Implementation of $f = \Sigma m\ (1, 2, 5, 6)$

Implement the following functions using a 3 to 8 line decoder.

$$f_1(a, b, c) = \Sigma m\ (0, 4, 6, 7)$$
$$f_2(a, b, c) = \Sigma m\ (1, 4, 5)$$

Solution: The minterms require for each function are ORed to realize the functions as shown he

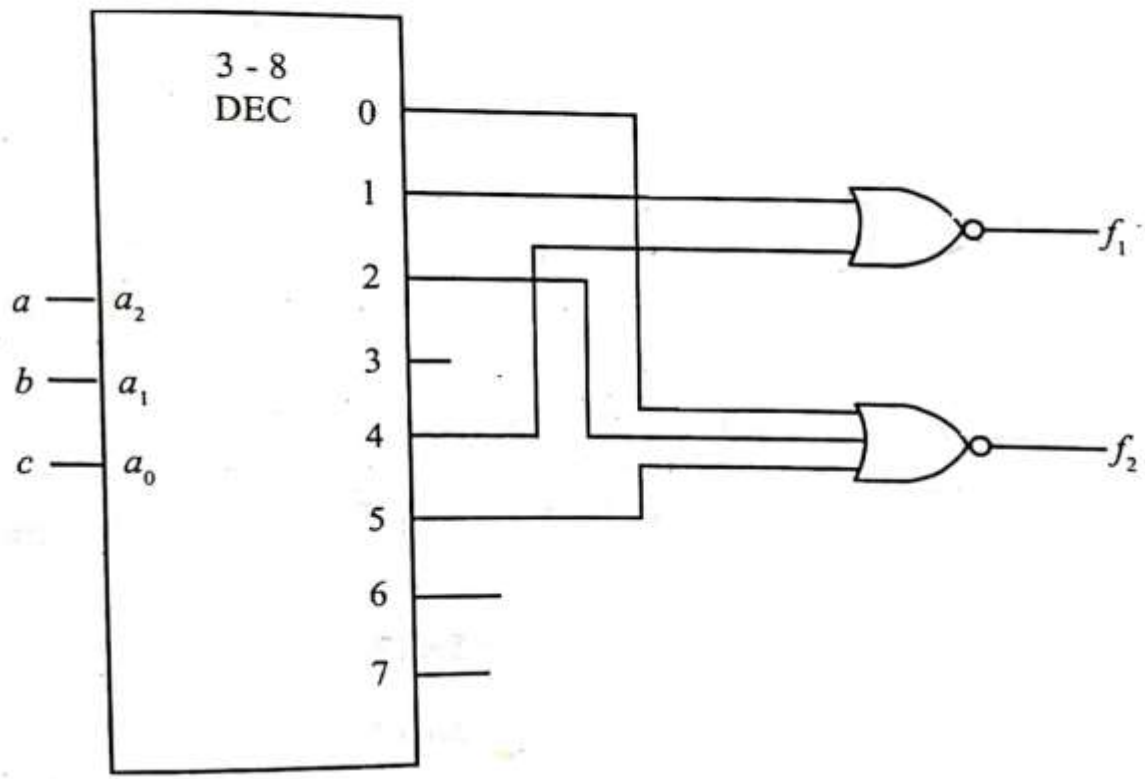


Implement the following functions using a decoder minimizing the number of inputs to be summed.

$$f_1(a, b, c) = \Sigma m\,(0, 2, 3, 5, 6, 7)$$
$$f_2(a, b, c) = \Sigma m\,(1, 3, 4, 6, 7)$$

$$f_1 = \Sigma m\,(0, 2, 3, 5, 6, 7)$$
$$\overline{f_1} = \Sigma m\,(1, 4)$$
$$f_2 = \Sigma m\,(1, 3, 4, 6, 7)$$
$$\overline{f_2} = \Sigma m\,(0, 2, 5)$$

The implementation is shown below.



Implement the following functions expressed in maxterm canonical form in two possible ways using a 3 to 8 decoder.

$$f_1(a, b, c) = \Pi M(2, 3, 4, 5, 7)$$
$$f_2(a, b, c) = \Pi M(0, 2, 4, 6, 7)$$

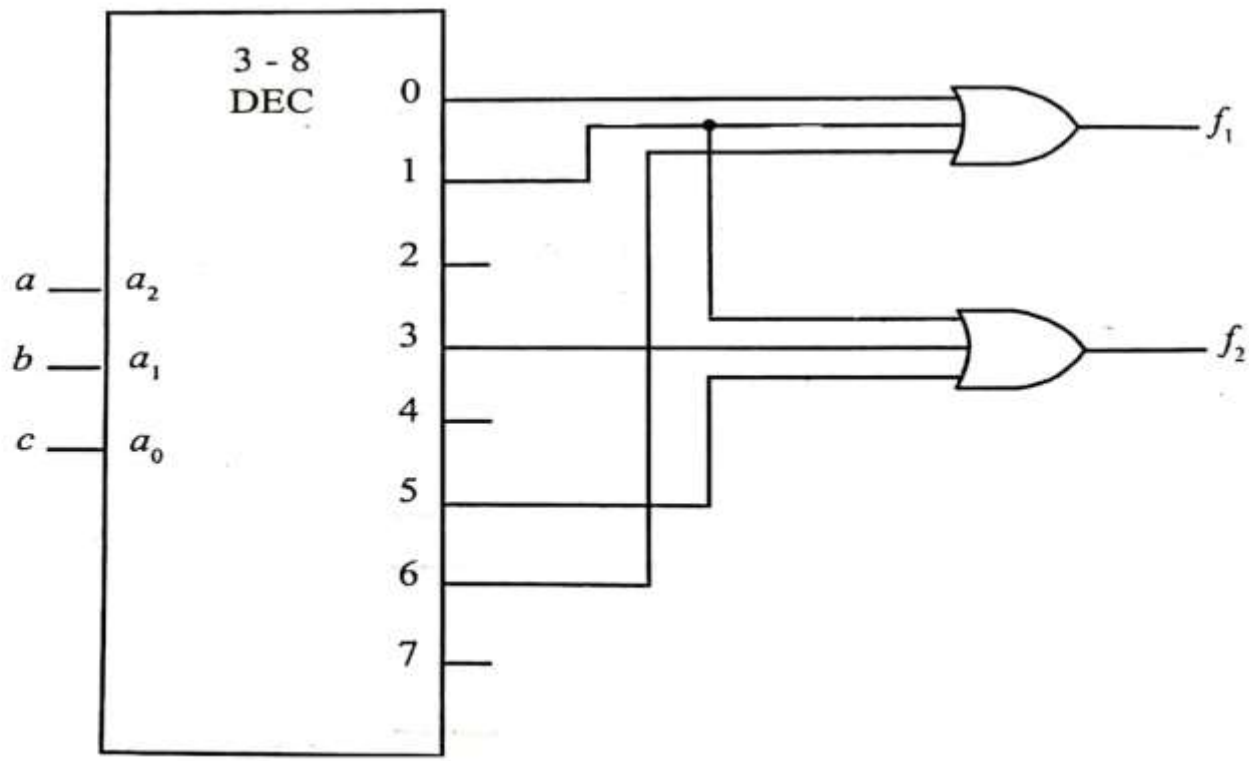
***Solution*:** We have seen in section 1.6.7 the method to convert from one canonical form to another we can express f_1 and f_2 in their minterm canonical form using m-notation as

$$f_1 = \Sigma m(0, 1, 6)$$

and

$$f_2 = \Sigma m(1, 3, 5)$$

The implementation using 3 to 8 line decoder is shown below.

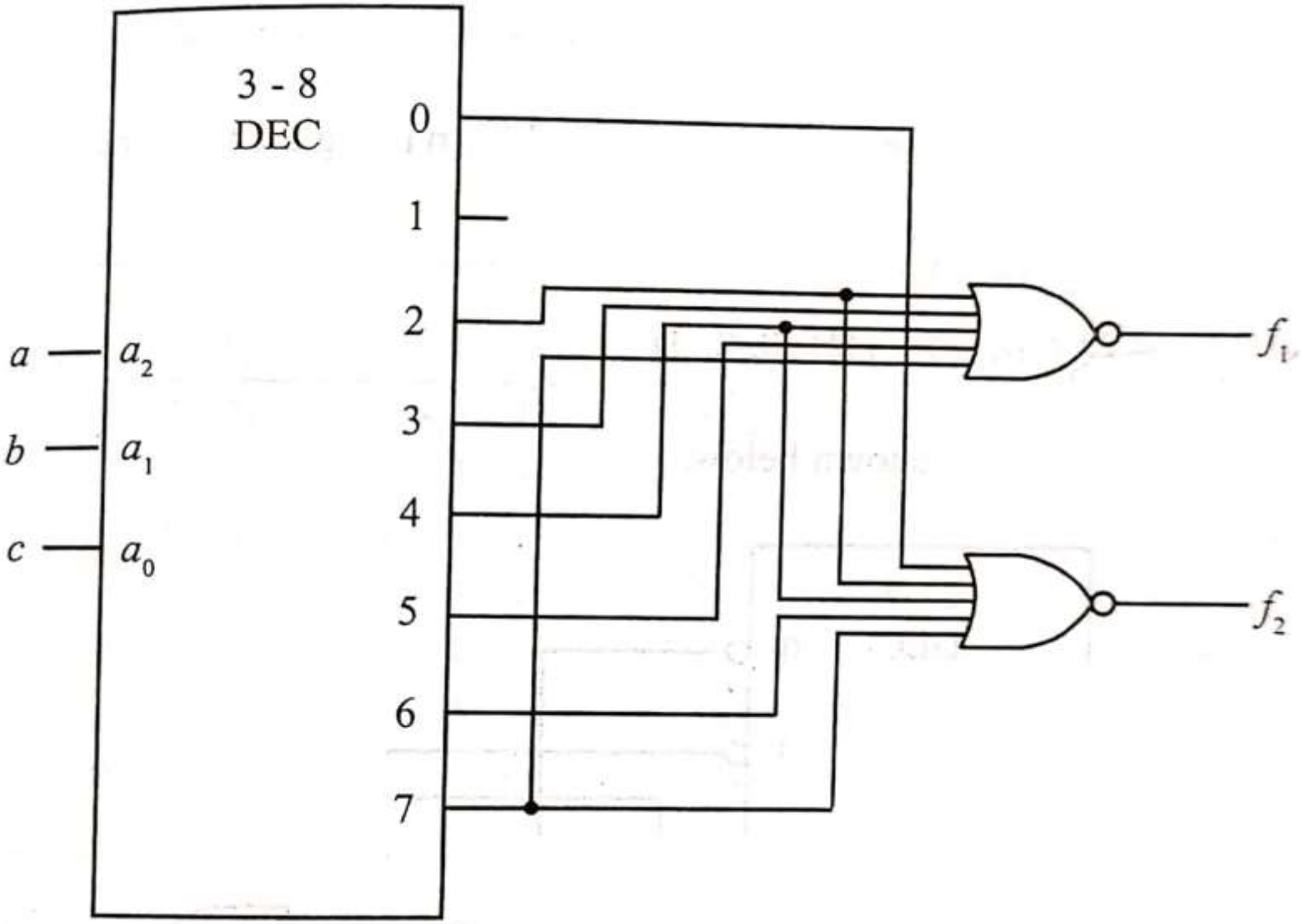


We can also implement these functions by implenting $\overline{f_1}$ and $\overline{f_2}$ by replacing the OR gate by a NOR gate.

$$f_1 = \Sigma m\,(0, 1, 6) \qquad \therefore \qquad \overline{f_1} = \Sigma m\,(2, 3, 4, 5, 7)$$

$$f_2 = \Sigma m\,(1, 3, 5) \qquad \therefore \qquad \overline{f_2} = \Sigma m\,(0, 2, 4, 6, 7)$$

Let us implement $\overline{f_1}$ and $\overline{f_2}$ and invert using NOR gate.

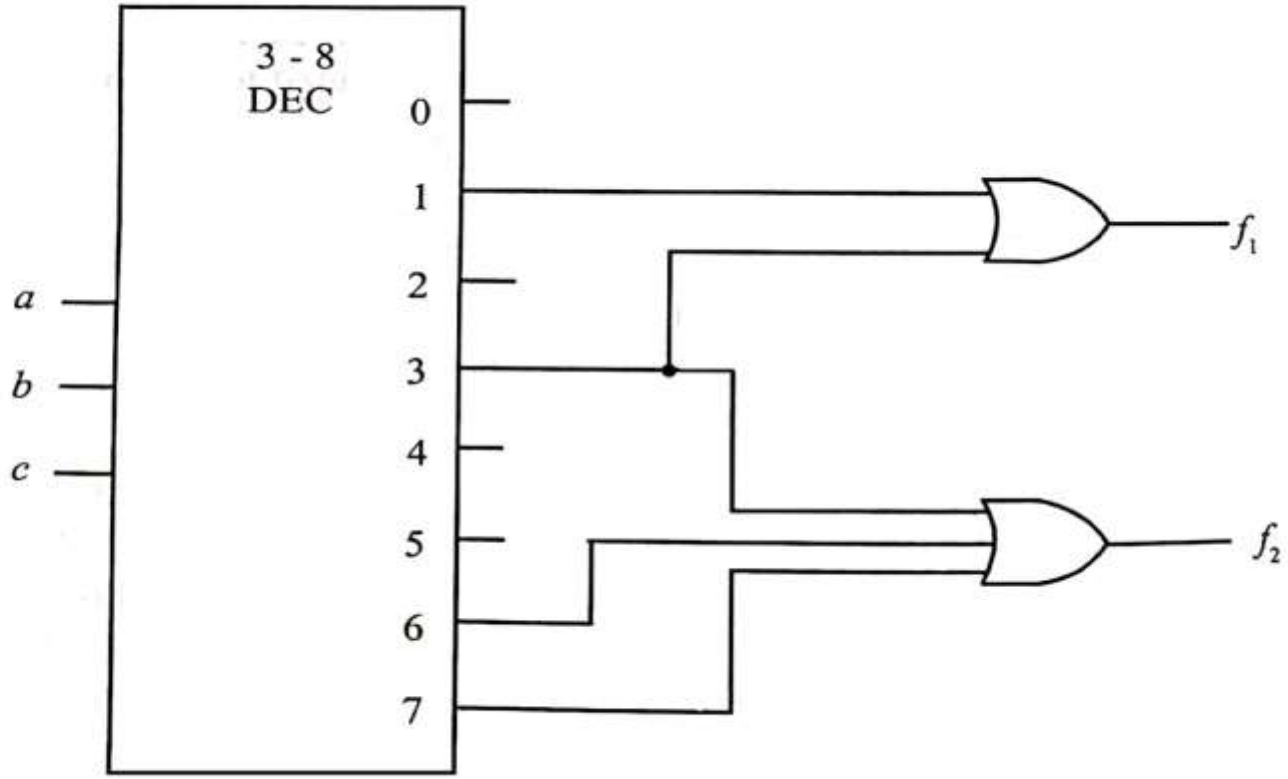

3 - 8
DEC
0
1
2
3
4
5
6
7
a
b
c
a_2
a_1
a_0
f_1
f_2

Using OR gates along with a decoder with uncomplemented outputs, implement following functions pair with minimal gate inputs.

$$f_1(a, b, c) = \Sigma m(1, 3)$$
$$f_2(a, b, c) = \Sigma m(3, 6, 7)$$

***Solution*:** Since the numbers of minterms are less than half the total number of minterms for a three variable function these functions can be directly implemented.

three



Implement the following function pairs using a decoder with uncomplemented outputs.

a. $f_1(a, b, c) = \Sigma m(0, 1, 5, 6, 7)$
$f_2(a, b, c) = \Sigma m(1, 2, 3, 6, 7)$

b. $f_1(a, b, c) = \Sigma m(0, 2, 4)$
$f_2(a, b, c) = \Sigma m(1, 2, 4, 5, 7)$
Implement for minimum gate inputs.

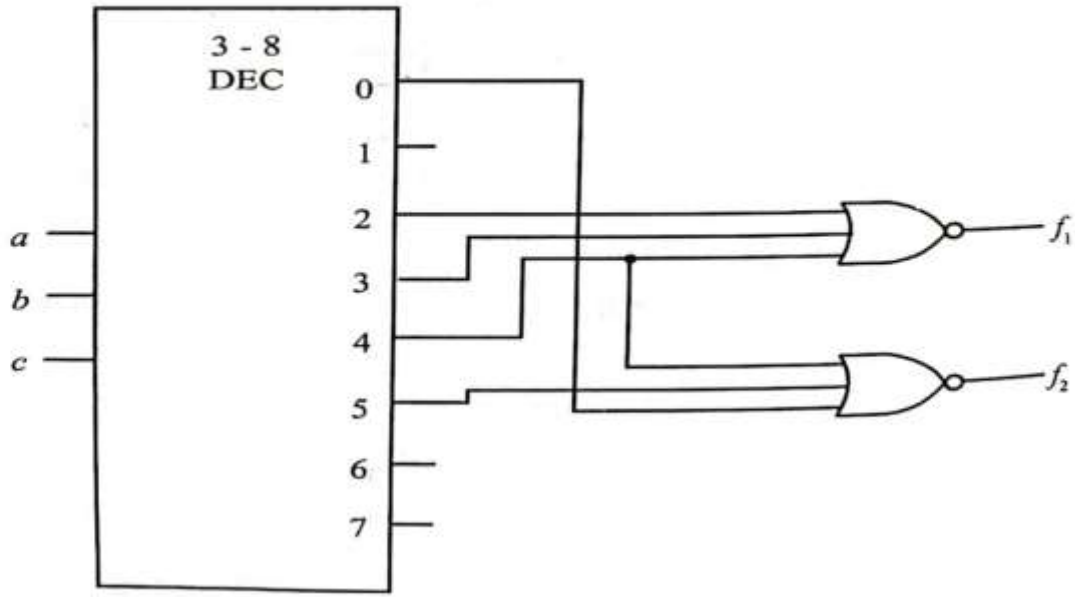
***Solution*:**

a. $f_1 = \Sigma m(0, 1, 5, 6, 7)$
$f_2 = \Sigma m(1, 2, 3, 6, 7)$

Here we need to implement $\overline{f_1}$ and $\overline{f_2}$ minimize the number of gate inputs.

$$\therefore \quad \overline{f_1} = \Sigma m(2, 3, 4)$$
$$\overline{f_2} = \Sigma m(0, 4, 5)$$

and replace the OR gate by a NOR gate.



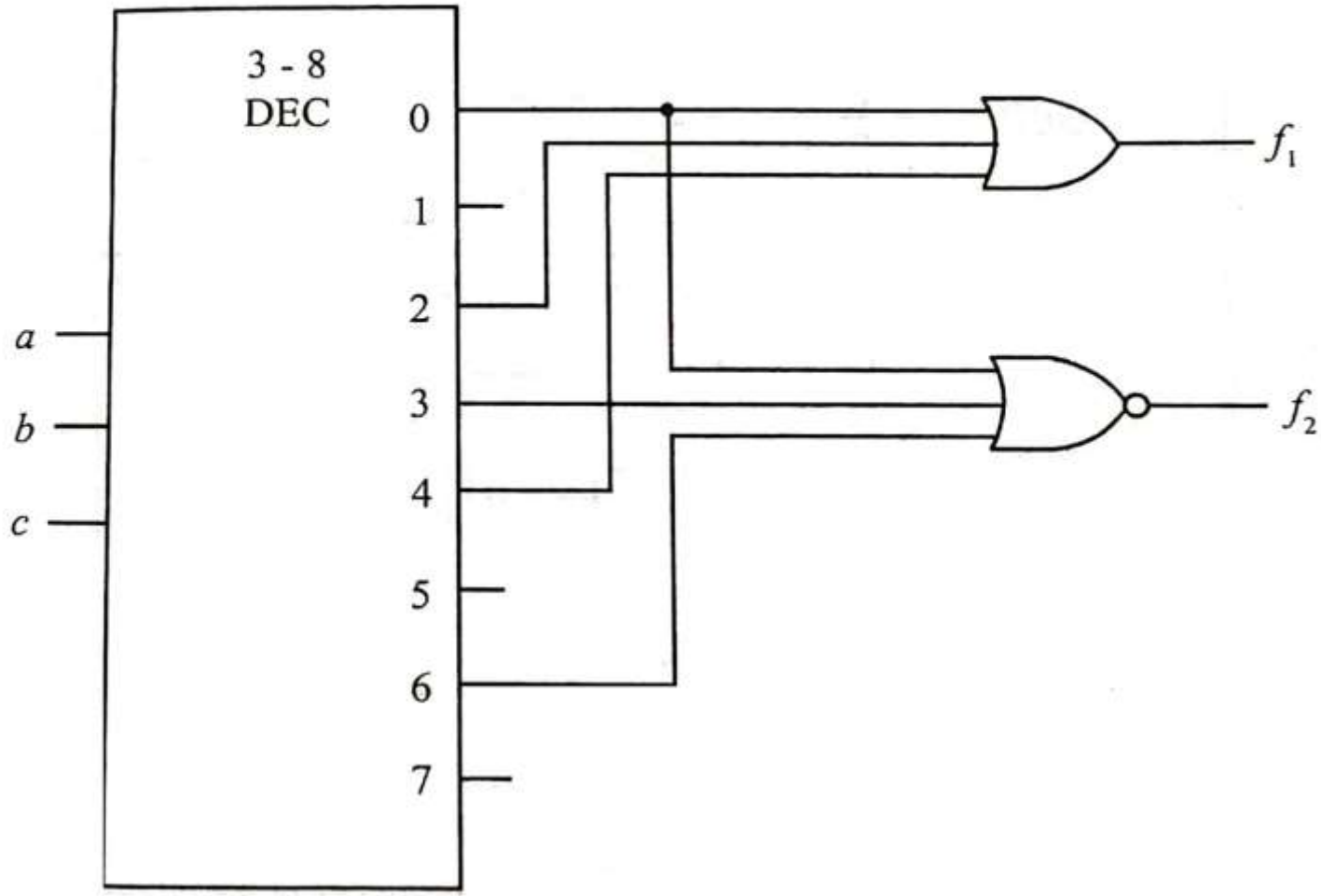
$$f_1 = \Sigma m(0,2,4)$$

$$f_2 = \Sigma m(1,2,4,5,7)$$

f_1 can be directly implemented

f_2 is realized by implementing $\overline{f_2}$ and inverting the output.

$$\overline{f_2} = \Sigma m\ (0, 3, 6)$$



Decoder

Implement the given function using a decoder

1. $B = f(a, b, c) = \sum(2, 4, 5, 7)$
2. Full adder sum and carry
3. Full subtractor difference and borrow
4. $C=f(w, x, y) = \prod(0, 1, 4, 7)$

Question Bank

- 1. Design half-adder and full-adder circuits using logic gates.
- 2. Design half-subtractor and full-subtractor using logic gates.
- 3. Explain the working of a 4-bit binary parallel adder with a relevant block diagram.
- 4. Explain the working of 4-bit binary parallel subtractor with relevant block diagram.
- 5. Explain the working of 4-bit binary parallel adder/subtractor using XOR gates with relevant block diagram.
- 6. Design 2-bit comparator using basic gates.
- 7. Explain octal to binary conversion using an encoder
- 8. Design 4 to 2 line encoder using basic gates.
- 9. Explain the working of 3-to-8-line decoder.
- 10. Design 3-to-8 line decoder using basic gates.
- 11. Draw truth table and logic diagram of 3 to 8 line decoder.
- 12. Design a full adder circuit using a decoder and 4:1 multiplexer.
- 13. Design 4 to 1 line multiplexer using basic gates.
- 14. Explain the working of a multiplexer with suitable example.
- 15. Explain the working of a multiplexer having 3 select lines.
- 16. Implement a full subtractor using 8:1 multiplexer.
- 17. Design a full adder circuit 4:1 multiplexer.

Assignment Questions –Part1

1. Explain the working of a 4-bit binary parallel adder/subtractor using XOR gates with a relevant block diagram and with an example.

2.a Explain octal to binary(8 to 3 line encoder) conversion using an encoder

b. Design a 4 to 2-line encoder using basic gates.

3. Explain the working of the 3-to-8-line decoder.

4. Design a 4-to-1-line multiplexer using basic gates.

5.Implement (a) $u = ad + b\bar{c} + bd$ using 8:1 MUX.

(b) F(A, B,C) =Σm(0,1,3,4) using 4:1 MUX and 8:1 MUX

6. Implement the given function using a decoder with a minimum number of gate inputs.

i) $F1 = \sum m(0,1,3)$

ii) $F2 = \sum m(0,2,3,5,7)$

iii) $F3 = \sum m\ (1,3,\ 5,6,\ 7)$

iv) $F4 = \pi M(1,2,3,5,6,7)$